



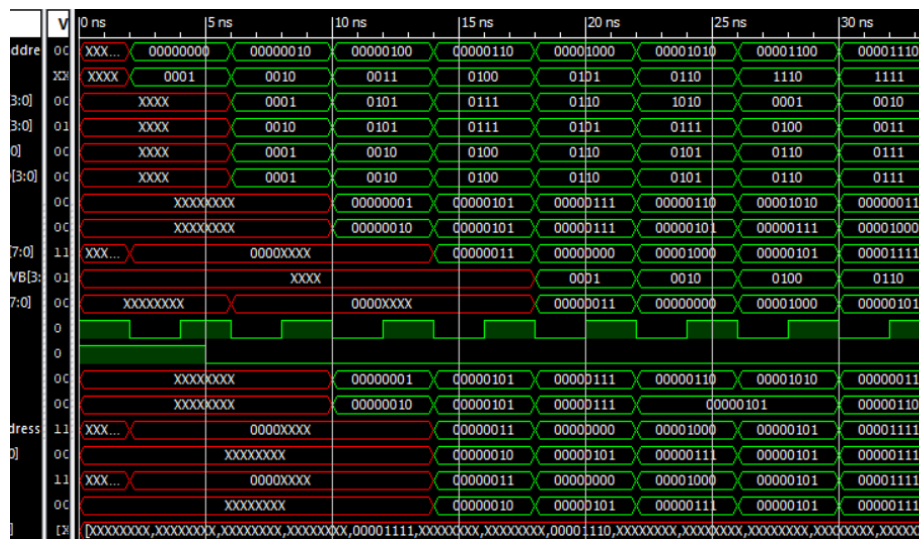
UNIVERSIDAD DE GUADALAJARA
CUCEI



Carrera: Ingeniería en Computación
Materia: Arquitectura de Computadoras

Reporte

PROYECTO FINAL - FASE 3



Integrantes:

Hernández Zúñiga Roberto López López Karla Valeria
Ortiz Segura Emmanuel Alejandro Segura Gutierrez Aaron Ricardo

Maestro: López Arce Delgado Jorge Ernesto

Sección: D03

Fecha: 23 mayo 2026

Índice

1. Introducción

1.1 Procesador MIPS

1.2 Tipos de Instrucciones

1.3 Instrucciones a Implementar

2. Objetivos

2.1 Objetivo General

2.2 Objetivos Específicos

3. Marco Teórico

3.1 Instrucciones Tipo R

3.2 Instrucciones Tipo I

3.2.1 Extensión de Signo (Sign Extend)

3.3 Instrucciones Tipo J

3.4 Operador Ternario

3.5 Desglose de Instrucciones Elegidas

3.6 Proceso de Compilación

3.7 Algoritmos Implementables con ISA

3.8 Búsqueda Binaria (Algoritmo)

3.9 Pipeline

4. Desarrollo

4.1 Pseudocódigo Algoritmo (Bubble Sort - Búsqueda Binaria)

4.2 Módulos Implementados (Verilog)

4.3 Ensamblador, Script Python y Binario

4.4 Resultados Simulación

4.4.1 Simulaciones Finales

5. Conclusión

6. Bibliografía

1. Introducción

1.1 Procesador MIPS

En términos sencillos, el procesador MIPS es como un "minimalista" de la tecnología, la esencia del diseño RISC. Su filosofía se basa en que menos es más, al reducir la complejidad de las tareas, el chip puede trabajar a una mayor velocidad.

Puntos clave del diseño

Gestión de Memoria Selectiva:

A diferencia de otros procesadores que se distraen buscando datos en la memoria RAM constantemente, MIPS es muy estricto. Solo usa dos comandos específicos para mover datos (lw-Load word y sw-Store word). Todo el "trabajo sucio" de cálculos ocurre internamente en sus 32 registros, lo que evita cuellos de botella.

Instrucciones Uniformes:

Imaginemos que todas las herramientas en una caja tuvieran exactamente el mismo tamaño y peso. Eso es MIPS, con todas sus instrucciones de 32 bits. Esto permite que el procesador no pierda tiempo midiendo o analizando cada comando; simplemente los recibe y los ejecuta de forma rítmica.

Trabajo en Cadena (Pipelining):

Su estructura está diseñada para funcionar como una línea de ensamblaje industrial. Mientras una instrucción se está terminando de escribir, otra ya se está procesando y una nueva está entrando al procesador. No hay tiempos muertos.

El "Ancla" del Cero:

Tiene una característica curiosa pero brillante: el registro cero. Es un valor constante que nunca cambia, manteniendo siempre un valor de 0, lo que sirve como punto de referencia rápido para inicializar valores, mover y limpiar datos o realizar comparaciones lógicas sin esfuerzo extra.

Las etapas clásicas de un procesador MIPS son:

1. **IF (Instruction Fetch):** Traer la instrucción de memoria.
2. **ID (Instruction Decode):** Decodificar y leer registros.
3. **EX (Execute):** Realizar la operación en la ALU (Unidad Aritmético Lógica).
4. **MEM (Memory Access):** Acceder a la RAM si es necesario.
5. **WB (Write Back):** Escribir el resultado en el registro.

1.2 Tipos de instrucciones

En el diseño de procesadores, existen 3 tipos de instrucciones que facilitan la decodificación en la UC, con formato de 32 bits, se describen de la siguiente forma:

Tipo R (registro)

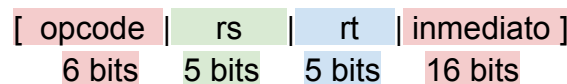
Para las operaciones aritmético-lógicas entre registros, por ejemplo, add y sub. Los 32 bits de la instrucción se dividen de la siguiente manera:



- **rs, rt:** Registros fuente.
- **rd:** Registro destino.
- **funct:** Código funcional que especifica la operación.
- Se decodifican los campos y se envían **rs** y **rt** a la ALU, el resultado de la operación especificada en **function** se escribe en la dirección de memoria **rd**.

Tipo I (inmediato)

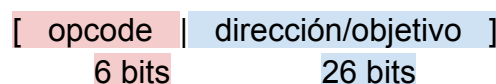
Para operaciones con operandos inmediatos (constantes), cargas/almacenamientos de memoria. Por ejemplo, sw (save word) y lw (load word). Su formato de 32 bits es el siguiente:



- **rs:** Registro fuente.
- **rt:** Registro destino o fuente (depende de la instrucción).
- **inmediato:** Valor constante de 16 bits, a menudo extendido con signo.
- El valor **inmediato** se extiende en signo y se utiliza junto con **rs** en la ALU.

Tipo J (salto/jump)

Para saltos incondicionales a direcciones de memoria. Por ejemplo, j(jump). El formato de instrucción a 32 bits es:



- **dirección:** Junto con el PC (Program Counter), define dirección de salto.

La unidad de control actualiza el PC directamente utilizando la **dirección** de 26 bits.

A continuación, se presenta el set de instrucciones que se utilizará en el proyecto.

1.3 Instrucciones a Implementar

Instrucción	Tipo	Sintaxis	Descripción
add 	R	add \$rd, \$rs, \$rt	Suma: $d = s + t$
sub 	R	sub \$rd, \$rs, \$rt	Resta: $d = s - t$
or 	R	or \$rd, \$rs, \$rt	Operación OR lógica: $d = s t$
and 	R	and \$rd, \$rs, \$rt	Operación AND lógica: $d = s \& t$
slt 	R	slt \$rd, \$rs, \$rt	Si $s < t$, $d = 1$; si no, $d = 0$
nop 	R	nop \$rd, \$rs, \$rt	No Operation: (equivale a sll \$0, \$0, 0)
addi 	I	addi \$rd, \$rs, constante	Suma inmediata: $d = s + \text{constante}$
subi 	I	addi \$rd, \$rs, -constante	Se usa addi con valor negativo
ori 	I	ori \$rd, \$rs, constante	OR inmediato: $d = s \text{constante}$
andi 	I	andi \$rd, \$rs, constante	AND inmediato: $d = s \& \text{constante}$
slti 	I	slti \$rd, \$rs, constante	SLT inmediato: Si $s < \text{constante}$, $d = 1$
lw 	I	lw \$rd, desplazamiento(\$rs)	$rd = rs + \text{desplazamiento}$
sw 	I	sw \$rd, desplazamiento(\$rs)	$rs + \text{desplazamiento} = rd$
beq 	I	beq \$rs, \$rt, etiqueta	Branch if Equal: Salta si $s = t$
bne 	I	bne \$rs, \$rt, etiqueta	Branch if Not Equal: Salta si $s \neq t$
bgtz 	I	bgtz \$rs, etiqueta	Branch if Greater Than Zero: Salta si $s > 0$
j 	J	j etiqueta	Salto incondicional a dirección de etiqueta
xor 	R	xor \$rd, \$rs, \$rt	$d = 1$, si los bits de s y t son diferentes
teq 	R	teq \$rs, \$rt	Excepción si el contenido de $s = t$
tge 	R	tge \$rs, \$rt	Excepción si $s \geq t$
sra 	R	sra \$rd, \$rt, shamt	Desplaza bits de t a la derecha shamt veces, manteniendo el bit de signo
srl 	R	srl \$rd, \$rt, shamt	Desplaza los bits de t a la derecha shamt veces y rellena dependiendo del signo

-  Instrucciones Aritmeticas y Logicas
-  Instrucciones con Inmediatos
-  Instrucciones de Memoria (Carga y Almacenamiento)
-  Instrucciones de Control de Flujo (Saltos)
-  Instrucciones de Desplazamiento
-  Instrucciones de Excepción (Trap)

2. Objetivos

2.1 Objetivo general

Explicar el funcionamiento del Datapath e implementar el diseño, tomando en cuenta todos los módulos necesarios para su funcionamiento, tales como, el banco de registros, ALU, multiplexores, unidades de control, desde la investigación, organización, división de tareas, implementación y ejecución, utilizando para esta primera fase las instrucciones de tipo R especificadas en la tabla descrita dentro de la introducción.

2.1 Objetivos específicos

- Investigar de manera profunda cada concepto para su entendimiento y aplicación.
- Desarrollar el código de prueba en ensamblador para instrucciones tipo R.
- Hacer las modificaciones necesarias a los módulos anteriormente diseñados para ejecutar todas las instrucciones tipo R de la tabla.
- Hacer el script de Python para convertir el archivo de código ensamblador a binario.
- Realizar los Test Bench necesarios para comprobar el funcionamiento de la práctica.

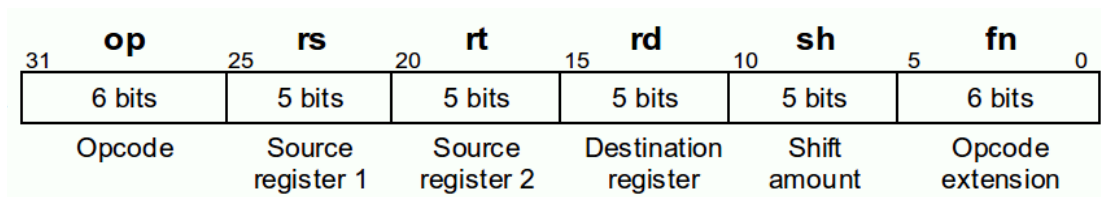
3. Marco Teórico

3.1 Instrucción Tipo R

En la arquitectura MIPS, las instrucciones tipo R (Register-type) se utilizan para realizar operaciones entre registros. Este tipo de instrucción se caracteriza por operar exclusivamente con datos almacenados en registros internos del procesador, sin acceso directo a la memoria.

El formato (máquina) de instrucción tipo R se compone de los siguientes campos:

- **opcode:** código de operación (generalmente 0 en tipo R)
- **rs:** primer registro fuente
- **rt:** segundo registro fuente
- **rd:** registro destino
- **shamt:** desplazamiento (shift amount)
- **funct:** especifica la operación exacta



Representación de la separación de bits para cada campo.

Como se observa en la imagen, de los 32 bits provenientes de la instrucción el campo function y el opcode necesitan de 6 bits para llevar a cabo la operación, y los campos restantes constan de 5 bits.

De acuerdo a esta información, la separación de campos en verilog para instrucciones del tipo R en formato MIPS, según los bits especificados para cada apartado, quedaría algo así:

```
assign opcode = instruccion[31:26];
assign rs     = instruccion[25:21];
assign rt     = instruccion[20:16];
assign rd     = instruccion[15:11];
assign shamt  = instruccion[10:6];
assign funct  = instruccion[5:0];
```

Opcode y **funct** con 6 bits cada uno, **rs**, **rt**, **rd** y **shamt** con 5 bits, dando el total de 32 bits para la instrucción.

Sintaxis para Ensamblador

La sintaxis básica para instrucciones tipo R (Registro-Registro) en lenguaje ensamblador, es la siguiente:

`opcode registro_destino , registro_fuente1 , registro_fuente2`

Ejemplo:

`add $7 , $4 , $2`

Ejemplo de la sintaxis para una suma en código ensamblador, se usa el operador `add`. Al número en la dirección de memoria 4 se le suma el número en la dirección de memoria 2, el resultado se escribe en la dirección de memoria 7.

Elementos de la Sintaxis

- **Mnemónico (Opcode):** Nombre de la instrucción (ej. ADD, SUB, AND, OR, SLT).
- **Registro Destino (rd):** El primer operando tras el mnemónico, donde se almacenará el resultado.
- **Registro Fuente 1 (rs):** Segundo operando, primer valor de entrada a la ALU.
- **Registro Fuente 2 (rt):** Tercer operando, segundo valor de entrada a la ALU.

Operaciones implementables

Existe una gran cantidad de operaciones aritmético-lógicas a realizar con las instrucciones tipo R, tales como `add`(suma), `sub`(resta), `and`, `or`. Dentro de estas una de las operaciones más relevantes es **SLT** (Set on Less Than). Esta instrucción compara dos registros y establece el valor del registro destino dependiendo del resultado de la comparación:

- Si **rs < rt**, entonces **rd = 1**
- En caso contrario, **rd = 0**

La instrucción SLT es fundamental para la implementación de estructuras de control en bajo nivel, como condicionales (`if`) y ciclos (`while`), ya que permite realizar comparaciones entre valores.

Internamente la ALU no puede realizar la comparativa entre un valor y otro e indicar si es menor o mayor directamente, es por eso que, esta operación suele implementarse mediante una resta (**rs - rt**) y la evaluación del bit de signo del resultado.

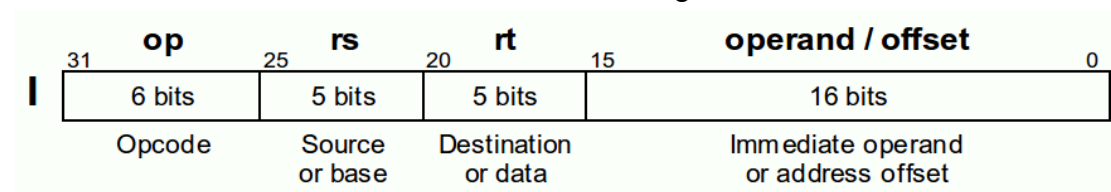
3.2 Instrucciones Tipo I

Las instrucciones Tipo I (Immediate-type) son fundamentales en la arquitectura MIPS para realizar operaciones que involucran operandos constantes (inmediatos) y para gestionar la transferencia de datos entre los registros del procesador y la memoria principal (RAM) . A diferencia de las instrucciones puramente aritméticas que operan sobre datos preexistentes en registros, el formato Tipo I dota al procesador de la capacidad de introducir nuevos valores al sistema y de acceder a estructuras de datos masivas guardadas en memoria.

El formato Tipo I mantiene la longitud estándar de 32 bits, pero redistribuye sus campos para albergar un número constante (inmediato). Los campos se dividen de la siguiente manera:

- **Opcode** (Código de Operación): 6 bits que definen la instrucción exacta (ej. lw, sw, addi).
- **rs** (Register Source): 5 bits que indican el primer registro fuente. En operaciones de memoria, este registro contiene la "dirección base".
- **rt** (Register Target): 5 bits que indican el registro destino (donde se guardará el resultado) o el registro fuente secundario (cuyo valor se guardará en memoria, como en el caso de sw) .
- **Inmediato** (Constante / Offset) [15:0]: 16 bits que representan un valor numérico directo o un desplazamiento (offset).

La distribución de los bits se estructura de la siguiente manera:



Diferencias Principales con las Instrucciones Tipo R

La transición del formato Tipo R al Tipo I implica modificaciones sustanciales en la decodificación lógica :

1. **Ausencia del campo funct:** Las instrucciones Tipo I carecen de los 6 bits finales de función. El opcode principal es el único responsable de indicarle a la Unidad de Control qué operación matemática debe forzar en la ALU.

2. **Destino del Registro:** En el formato Tipo R, el resultado siempre se almacena en el registro rd (bits 15:11). En el formato Tipo I, al no existir el campo rd, el registro de destino pasa a ser rt (bits 20:16). Esto obliga al hardware a implementar un multiplexor (RegDst) para decidir qué bits definen la dirección de escritura.
3. **Operando Inmediato:** Se sacrifica el tercer registro para permitir la incrustación de una constante de 16 bits.

Para el correcto funcionamiento de estas instrucciones, necesitamos agregar un pequeño módulo llamado “Sign extend” o extensor de signo, el cual profundizaremos más en un momento.

Sintaxis en Ensamblador

La sintaxis básica para instrucciones tipo I (inmediatas) en lenguaje ensamblador, es la siguiente:

opcode registro_destino , registro_fuente , valor_inmediato

Ejemplo:

addi \$7 , \$4 , #2

Ejemplo de la sintaxis para una suma inmediata en código ensamblador, se usa el operador addi. Al número en la dirección de memoria 4 se le suma el número 2 (constante), el resultado se escribe en el registro 7.

Extensión de Signo (Sign Extend)

La Unidad Aritmético Lógica (ALU) del procesador MIPS está diseñada para operar exclusivamente con datos de 32 bits de longitud. Dado que el campo inmediato de la instrucción Tipo I aporta únicamente 16 bits, el hardware se enfrenta a una incompatibilidad de tamaño.

Para resolver esto sin alterar el valor matemático (especialmente si el número es negativo en complemento a dos), se implementa un módulo combinacional denominado Extensor de Signo (Sign Extend). Este módulo toma los 16 bits del inmediato y replica el bit más significativo (el bit 15, correspondiente al signo) 16 veces hacia la izquierda para completar una palabra de 32 bits .

En Verilog, esta operación se logra de forma eficiente mediante la concatenación:

```
assign out = {{16{in[15]}}, in};
```

Instrucciones de Memoria (Lw - Sw)

Las instrucciones lw y sw utilizan el formato Tipo I para implementar el concepto de direccionamiento base más desplazamiento. Lw significa "load word", y en este contexto lo que carga es un dato, de nuestra memoria al banco de registros, su propósito general, es obtener variables para hacer operaciones matemáticas o lógicas. Sw significa store word y hace lo contrario a lw, guarda el registro que tenemos en nuestra memoria de datos, su propósito general es guardar resultados finales o valores temporales en la memoria.

La sintaxis para estas dos instrucciones es la siguiente:

lw \$rt, desplazamiento(\$rs) : Carga una palabra (32 bits) de la memoria hacia el registro rt.

sw \$rt, desplazamiento(\$rs) : Guarda la palabra del registro rt en la memoria.

Instrucción Beq

La instrucción beq (Branch if Equal) en MIPS es de tipo I (Inmediato) y se utiliza para tomar decisiones. Compara el contenido de dos registros; si son iguales, realiza un salto a una etiqueta; si no, continúa con la siguiente instrucción.

Funcionamiento:

1. El procesador calcula la dirección de destino de la siguiente forma: Se incrementa el Contador de Programa (PC) para que apunte a la instrucción siguiente.
2. El campo "Inmediato" (offset) se extiende de signo a 32 bits y se multiplica por 4 (ya que en MIPS las instrucciones están alineadas a palabras de 4 bytes).
3. Si los registros son iguales, la nueva dirección de ejecución será:
$$PC_nueva = (PC + 4) + (Inmediato \times 4).$$

Si no son iguales, simplemente se ejecuta $PC+4$.

3.3 Instrucciones Tipo J

Las instrucciones Tipo J (Jump) se utilizan en la arquitectura MIPS para realizar saltos incondicionales hacia una dirección específica de la memoria de

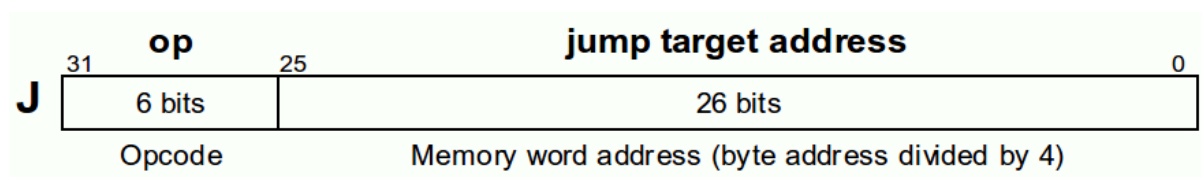
instrucciones. A diferencia de las instrucciones de salto condicional Tipo I (como beq o bne), que dependen del resultado de una comparación en la ALU para decidir si saltan o no, las instrucciones Tipo J rompen la secuencialidad del programa de forma obligatoria y directa cada vez que son ejecutadas.

Su propósito principal es permitir la transferencia del control del flujo del programa hacia subrutinas, funciones o etiquetas lejanas que se encuentran fuera del rango de alcance de los saltos relativos de Tipo I.

Formato de Máquina (Codificación de Bits)

El formato de una instrucción Tipo J consta de 32 bits en total. Para maximizar la cantidad de memoria a la que se puede saltar, este formato prescinde por completo del uso de registros y divide sus campos en únicamente dos apartados:

- **Opcode** (Código de Operación): Ocupa 6 bits y define la operación específica de salto. Para la instrucción estándar j, el código binario asignado es 000010.
- **Target Address** (Dirección Objetivo) [25:0]: Ocupa los 26 bits restantes de la instrucción. Contiene la dirección codificada a la que el procesador debe desviar el Program Counter (PC).



Para saltar a una dirección de 32 bits, el procesador reconstruye la posición destino de la siguiente manera:

1. Toma los 26 bits del campo de dirección y los desplaza 2 posiciones a la izquierda (lo que equivale a multiplicar por 4).
2. Toma los 4 bits más significativos del registro Contador de Programa (PC +4).
3. Concatena ambos para formar la dirección final de 32 bits a la que saltará el procesador.

Diferencias Principales con las Instrucciones Tipo R

Es fundamental contrastar el formato Tipo J con el formato Tipo R trabajado en la primera fase para comprender la optimización del hardware:

- **Uso de Registros:** Las instrucciones Tipo R operan exclusivamente con datos almacenados en los registros internos (rs, rt, rd). Las instrucciones Tipo J no interactúan con el Banco de Registros; operan directamente sobre el Program Counter (PC) utilizando un valor inmediato.
- **Especificación de la Operación:** En el formato Tipo R, el opcode general suele ser `000000`, y la operación exacta de la ALU se determina mediante los 6 bits del campo funcional (funct) al final de la instrucción. En el formato Tipo J, el opcode de 6 bits define por sí mismo y de manera absoluta la acción de salto, eliminando la necesidad de un campo `funct`.
- **Uso de los Bits:** Tipo R divide sus 32 bits en 6 campos pequeños para direccionar múltiples registros y desfaseamientos (shamt). Tipo J fusiona 26 bits en un solo bloque para disponer del mayor espacio posible para una dirección de memoria.

3.4 Operador Ternario

La operación ternaria es común en lenguajes de programación de alto nivel como C y Verilog. Se trata de una forma compacta de expresar una estructura condicional.

La sintaxis de este operador en C sería la siguiente:

condición ? valor_si_verdadero : valor_si_falso;

Esta operación evalúa una condición lógica. Si la condición es verdadera, retorna el primer valor, si es falsa, retorna el segundo.

En el contexto de diseño digital (por ejemplo, en Verilog), la operación ternaria se utiliza frecuentemente para describir lógica combinacional. Por ejemplo:

assign salida = (a < b) ? 1 : 0;

Aquí se observa una equivalencia directa con la instrucción SLT, ya que se está evaluando una condición de comparación y asignando un valor binario como resultado. Es útil, ya que simplifica el código y puede representar directamente multiplexores en hardware, lo que facilita la síntesis.

3.5 Desglose de Instrucciones Elegidas

Las instrucciones xor, sra y srl, son lógicas y de desplazamiento, estas instrucciones operan a nivel de bits o desplazan la posición de los mismos dentro de un registro.

- **XOR** (lógica): Operación or exclusiva bit a bit entre los valores de 2 registros y almacena el resultado en un tercer registro.
- **SRL** (lógica): Siempre rellena con 0.
- **SRA** (aritmética): Rellena con el valor del bit de signo (si el número era negativo, rellena con 1; si era positivo, con 0) para preservar el valor matemático.

Las instrucciones como `teq` y `tge`, de tipo R, comparan dos registros. Si la condición se cumple, el procesador genera una excepción (interrupción de software), lo que suele detener el programa o saltar a un manejador de errores.

- **TEQ** (excepción): Compara dos registros y genera una excepción (trampa) si son iguales, se utiliza para gestionar errores de software, como la división por cero, terminando el programa y transfiriendo el control al gestor de excepciones.
- **TGE** (excepción): Genera una trampa si $rs \geq rt$. Se usa comúnmente para verificar límites, depuración, validación de índices de arreglos.

3.6 Fases del Proceso de Compilación

Este es el proceso que transforma un programa escrito en un lenguaje de programación de alto nivel (código fuente) a un lenguaje de bajo nivel (código objeto o binario). A diferencia de un intérprete (que traduce línea por línea mientras el programa corre), el compilador traduce todo el programa antes de que se ejecute.

El transcurso desde que presionamos “compilar” hasta que obtienes un archivo `.exe` o un binario para MIPS, como será en esta práctica, se divide en etapas o fases:

❖ **Análisis Léxico** (Scanner)

El compilador lee el código fuente carácter por carácter y los agrupa en unidades con significado llamadas *tokens*. Por ejemplo: Si escribes `suma = 5 + 10`, el analizador identifica: un identificador (`suma`), un operador de asignación (`=`), un número (`5`), etc.

❖ **Análisis Sintáctico** (Parser)

Aquí se revisa que los *tokens* sigan las reglas gramaticales del lenguaje. Se crea una estructura jerárquica llamada Árbol de Sintaxis Abstracta (AST). Si te falta un punto y coma o un paréntesis, aquí es donde el compilador te lanza el error.

❖ **Análisis Semántico**

El compilador verifica que el código tenga sentido, por ejemplo, al sumar una palabra con un número, o usar una variable no declarada, si es así, el proceso se detiene.

❖ **Generación de Código Intermedio**

El compilador crea una versión del código que es independiente de la máquina. Es una representación simplificada que facilita las optimizaciones antes de pasar al lenguaje específico de un procesador.

❖ **Optimización de Código**

Esta es la fase "inteligente". El compilador intenta que el programa sea más rápido o ocupe menos memoria. Por ejemplo, si tienes una operación como $x = 2 * 4$, el optimizador la cambia directamente por $x = 8$ para ahorrarle trabajo al procesador.

❖ **Generación de Código de Salida**

El compilador traduce el código optimizado al lenguaje específico de la arquitectura destino (como las instrucciones de 32 bits de MIPS o de x86).

¿Qué pasa cuando se completaron las fases?

El compilador, no es el proceso completo. Después de compilar, se obtiene un "archivo objeto" (.o o .obj), pero este aun no funciona solo.

El **Linker** o enlazador toma el archivo objeto y lo une con las librerías del sistema (por ejemplo, las que permiten imprimir en pantalla o leer el teclado) para generar el ejecutable final.

Entonces se inicia por un **análisis**, entender que escribió el programador y detectar errores de escritura, pasa por una **síntesis**, construir el programa en lenguaje máquina y optimizarlo, y finalmente llegamos al **enlazado**, unir todas las piezas y librerías en un solo archivo funcional.

3.7 Algoritmos Implementables con la ISA

De acuerdo al set de instrucciones definido para este proyecto, se pueden llegar a construir algoritmos con cierta complejidad lógica y aritmética, combinando instrucciones de registro, inmediatas y saltos.

Estructuras de control de flujo (lógica condicional)

Con instrucciones de salto y comparación, podemos recrear cualquier lógica de decisión:

- **Sentencias if-else:** Utilizando `beq` y `bne` para saltar a diferentes bloques de código según si dos registros son iguales o no.
- **Bucles (for, while):** Combinando saltos condicionales (`bgtz`, `beq`) con saltos incondicionales (`j`) al final del bloque para repetir una secuencia.
- **Validaciones críticas:** Usando las instrucciones de excepción `teq` y `tge` para detener el programa si ocurre un error lógico (como un índice fuera de rango o una condición prohibida).

Manipulación de Estructuras de Datos

Con las instrucciones de carga y almacenamiento, puedes gestionar datos en la memoria RAM:

- **Recorrido de Arreglos (Arrays):** Utilizando `lw` (cargar) y `sw` (guardar) junto con `addi` para incrementar el puntero o índice de memoria.
- **Copia de datos:** Algoritmos para mover bloques de información de una dirección de memoria a otra.

Aritmética y Procesamiento de Bits

- **Cálculos Matemáticos:** Implementación de fórmulas complejas usando `add`, `sub` y `addi`.
- **Multiplicación y División por potencias de 2:** Usando `srl` y `sra` para desplazar bits, lo cual es mucho más rápido que realizar operaciones matemáticas completas.
- **Máscaras de Bits:** Con `and`, `or`, `xor` y `andi`, puedes extraer, encender o invertir bits específicos dentro de una palabra de datos (útil para protocolos de comunicación o control de hardware).

Algoritmos de Ordenamiento y Búsqueda

Es posible implementar algoritmos clásicos de ciencias de la computación:

- **Búsqueda Lineal:** Recorriendo la lista con `lw` y comparando con `beq`.
- **Ordenamiento** (como el método de la burbuja): Utilizando `slt` o `slti` para comparar valores y `sw` para intercambiar sus posiciones en memoria si uno es menor que otro.

3.8 Búsqueda Binaria (Algoritmo Implementado)

La búsqueda binaria es uno de los algoritmos de búsqueda más eficientes en informática, el método que utiliza es sencillo, ya que, para encontrar algo, no necesitas mirar cada elemento, solo necesitas saber hacia qué dirección moverte.

El algoritmo utiliza una estrategia de división sucesiva. En lugar de revisar uno por uno (como la búsqueda secuencial), este método divide el espacio de búsqueda a la mitad en cada paso. Para que la búsqueda binaria funcione, la colección de datos (arreglo o lista) debe estar ordenada (ya sea de forma ascendente o descendente). Si los datos están desordenados, el algoritmo fallará, ya que no podrá descartar mitades con seguridad.

Funcionamiento

Imagina que buscas un numero X en un arreglo:

- **Definir límites:** Se establecen dos punteros: *Inicio* (posición 0) y *Fin* (última posición).
- **Calcular el punto medio:** Se halla el índice central usando la fórmula:

$$Medio = \frac{Inicio + Fin}{2}$$

- **Comparar:**
 - Si el elemento en *Medio* es igual a X: La búsqueda termina.
 - Si el elemento en *Medio* es mayor que X: El número debe estar en la mitad izquierda. Entonces, movemos el puntero *Fin* a *Medio* - 1.
 - Si el elemento en *Medio* es menor que X: El número debe estar en la mitad derecha. Entonces, movemos el puntero *Inicio* a *Medio* + 1.
- **Repetir:** El proceso vuelve al paso 2 con los nuevos límites hasta encontrar el valor o hasta que Inicio supere a Fin (lo que significa que el valor no existe en la lista).

Lo que hace especial a este algoritmo es su velocidad. Mientras que una búsqueda lineal tarda un tiempo proporcional al número de elementos (n), la búsqueda binaria reduce el problema exponencialmente.

Ventajas	Desventajas
Búsqueda rápida en listas grandes.	Requiere datos pre-ordenados.
Consumo de recursos mínimo.	Menos eficiente en listas pequeñas.
Fácil de implementar recursivamente.	Insertar datos obliga a reordenar.

3.9 Pipeline

El pipeline (o segmentación) en Verilog es una técnica de diseño de hardware donde las operaciones se dividen en etapas secuenciales. Entre cada etapa se utilizan registros (Flip-Flops) para retener los datos. Esto permite procesar múltiples

instrucciones o datos de manera simultánea, mejorando drásticamente la frecuencia de reloj y el rendimiento.

Por ejemplo, si tenemos una tarea que requiere 3 pasos secuenciales: A, B y C.

- **Sin pipeline:** El sistema espera a que C termine con el primer dato antes de empezar con el paso A del segundo dato.
- **Con pipeline:** Se divide en 3 etapas separadas por registros. Cuando el primer dato pasa a la etapa B, un segundo dato puede empezar el paso A al mismo tiempo.

Elementos clave para Pipelining

- **Registros de Segmentación (Pipeline Registers):** Son los bloques de memoria (usualmente reg en Verilog) que separan una etapa de la siguiente. Almacenan los datos intermedios para sincronizarlos con el flanco de subida del reloj posedge clk.
- **Latencia (Latency):** El número de ciclos de reloj que tarda un solo dato o instrucción en atravesar todas las etapas del pipeline desde la entrada hasta la salida.
- **Rendimiento (Throughput):** La cantidad de resultados que produce el circuito por ciclo de reloj. En un pipeline ideal, se obtiene un resultado por cada ciclo.

Implementación dentro del proyecto

El procesador MIPS clásico utiliza una segmentación de cauce (pipeline) dividida en 5 etapas o fases, esto son 5 ciclos de reloj. Esta técnica permite procesar múltiples instrucciones simultáneamente al traslapar sus ciclos de ejecución.

Las 5 fases secuenciales son:

1. **Captura (Instruction Fetch - IF):** Extrae la instrucción de la memoria (apuntada por el Contador de Programa o PC) y la almacena en el registro temporal del pipeline.
2. **Decodificación (Instruction Decode - ID):** Interpreta la instrucción, extrae los valores necesarios desde el archivo de registros y genera las señales de control.
3. **Ejecución (Execute - EX):** La Unidad Aritmético-Lógica (ALU) realiza la operación matemática, lógica o calcula la dirección de memoria en el caso de instrucciones de salto o desplazamiento.

4. **Acceso a Memoria (Memory Access - MEM):** Se lee o escribe en la memoria de datos (solo se utiliza para instrucciones de carga lw o almacenamiento sw).
5. **Escritura de Retorno (Write Back - WB):** El resultado obtenido en las etapas anteriores se guarda o escribe de vuelta en el archivo de registros de la CPU.

Para la correcta implementación de un pipeline dentro de un procesador deben tenerse en cuenta los riesgos de pipeline (**Hazards**), el procesador debe manejar dependencias de datos (*data hazards*), de control (*branch hazards*) y conflictos estructurales, y de igual manera, las burbujas (**Stalls**), para evitar fallos, el procesador puede insertar "burbujas" o ciclos de espera (nop) cuando ocurren saltos condicionales o si una instrucción depende del resultado de la anterior que aún no está lista.

Forwarding Unit (módulo implementado)

La unidad de forwarding (también conocida en español como unidad de reenvío o cortocircuito) es un componente de hardware en procesadores con pipeline (segmentación). Su función principal es adelantar o pasar directamente los datos recién calculados a las instrucciones que los necesitan de inmediato, evitando tiempos de espera.

Sin forwarding, el procesador tendría que esperar a que la primera instrucción termine toda su ruta y guarde el resultado en la memoria, provocando un retraso llamado riesgo de datos (data hazard).

La unidad de forwarding detecta estas dependencias de datos. En lugar de esperar a que el dato llegue al registro final, el hardware captura el resultado temporal (por ejemplo, cuando sale de la etapa de Ejecución) y lo enruta a través de buses especiales directamente a la etapa donde la siguiente instrucción requiere ese operando.

4. Desarrollo

4.1 Pseudocódigo Algoritmo (Bubble Sort - Búsqueda Binaria)

ORDENAMIENTO BUBBLE SORT

```
para i = 0 hasta n-1
  para j = 0 hasta n-i-2
    si arreglo[j] > arreglo[j+1]
      intercambiar(arreglo[j], arreglo[j+1])
```

BÚSQUEDA BINARIA

(Inicialización de variables)

```
izq <- 0
der <- 0
Mientras NO (der < izq) Hacer:

  suma_indices <- izq + der
  medio <-
  Desplazar_Derecha_Bits (Suma_Indices 1)

  Offset_temp <- medio + medio
  Offset_final <- Offset_temp + Offset_temp

  Direccion_actual <- DireccionBase_A + Offset_final

  Si valor_leido == x Entonces:
    Retornar medio

  Si no, Si valor_leido < x Entonces:
    izq <- medio + 1

  Si no
    der <- medio - 1

  Fin Si
Fin Mientras
Retornar -1
```

4.2 Módulos Implementados (Verilog)

Módulo MIPS Top Module

```
module MIPS_TM (
    input clk,
    input reset,
    output [31:0] out_result
);
    // Cables del PC
    wire [31:0] pc_actual;
    wire [31:0] pc_siguiente;
    wire [31:0] pc_mas_4;
    wire [31:0] pc_branch;
    wire [31:0] pc_jump;

    // Cables de instrucciones y control
    wire [31:0] instr_to_dp;
    wire [31:0] ifid_instr;
    wire [31:0] ifid_pc4;
    wire branch_sig, zero_sig, pcsource;
    wire [31:0] sign_ext_val;
    wire jump_sig;
    wire [25:0] jump_target;
    wire [31:0] idex_pc4_branch; //PARA EL PC EN LA ETAPA EX

    PC pc1(
        .clk(clk),
        .reset(reset),
        .next_pc(pc_siguiente),
        .pc_out(pc_actual)
    );

    Mem_I memi(
        .dir(pc_actual),
        .DSalida(instr_to_dp)
    );

    BufferInstr ifid(
        .clk(clk),
        .reset(reset),
        .flush(pcsource | jump_sig), //Vacía el buffer si hay salto
        .instr_in(instr_to_dp),
        .pc4_in(pc_mas_4),
        .instr_out(ifid_instr),
        .pc4_out(ifid_pc4)
    );

    DPTR dp(
        .clk(clk),
        .reset(reset),
        .instruction(ifid_instr),
        .pc4_in(ifid_pc4),          //Manda el PC+4 al DPTR
    );
endmodule
```

```

        .result(out_result),
        .Branch_out(branch_sig),
        .Zero_out(zero_sig),
        .SignExt_out(sign_ext_val),
        .PC4_Branch_out(idex_pc4_branch), //Recibe el PC alineado de la etapa EX
        .Jump_out(jump_sig),
        .JumpTarget_out(jump_target)
    );

    // Sumador 1: PC + 4
    assign pc_mas_4 = pc_actual + 32'd4;

    // Sumador 2: Destino del Branch
    assign pc_branch = idex_pc4_branch + (sign_ext_val << 2);

    // Cálculo de la dirección del Jump
    assign pc_jump = {ifid_pc4[31:28], jump_target, 2'b00};

    // Mux del PC
    assign pc_siguiente = (reset) ? 32'b0 : (jump_sig == 1'b1) ? pc_jump : (pcsource == 1'b1) ?
pc_branch : pc_mas_4;

    // Compuerta AND para el Branch (lógica de inversión del BNE ya esta em DPTR)
    assign pcsource = (!reset) ? (branch_sig & zero_sig) : 1'b0;

endmodule

```

```

1  module MIPS_TM (
2      input clk,
3      input reset,
4      output [31:0] out_result
5  );
6      // Cables del PC
7      wire [31:0] pc_actual;
8      wire [31:0] pc_siguiente;
9      wire [31:0] pc_mas_4;
10     wire [31:0] pc_branch;
11     wire [31:0] pc_jump;
12
13     // Cables de instrucciones y control
14     wire [31:0] instr_to_dp;
15     wire [31:0] ifid_instr;
16     wire [31:0] ifid_pc4;
17     wire branch_sig, zero_sig, pcsource;
18     wire [31:0] sign_ext_val;
19     wire jump_sig;
20     wire [25:0] jump_target;
21     wire [31:0] idex_pc4_branch; //PARA EL PC EN LA ETAPA EX
22
23     PC pc1(
24         .clk(clk),
25         .reset(reset),
26         .next_pc(pc_siguiente),
27         .pc_out(pc_actual)
28     );
29
30     Mem_I memi(
31         .dir(pc_actual),
32         .DSalida(instr_to_dp)
33     );
34
35     BufferInstr ifid(

```

```

36     .clk(clk),
37     .reset(reset),
38     .flush(pcsource | jump_sig), //Vacía el buffer si hay salto
39     .instr_in(instr_to_dp),
40     .pc4_in(pc_mas_4),
41     .instr_out(ifid_instr),
42     .pc4_out(ifid_pc4)
43 );
44
45 DPTR dp(
46     .clk(clk),
47     .reset(reset),
48     .instruction(ifid_instr),
49     .pc4_in(ifid_pc4),           //Manda el PC+4 al DPTR
50     .result(out_result),
51     .Branch_out(branch_sig),
52     .Zero_out(zero_sig),
53     .SignExt_out(sign_ext_val),
54     .PC4_Branch_out(idex_pc4_branch), //Recibe el PC alineado de la etapa EX
55     .Jump_out(jump_sig),
56     .JumpTarget_out(jump_target)
57 );
58
59 // Sumador 1: PC + 4
60 assign pc_mas_4 = pc_actual + 32'd4;
61 // Sumador 2: Destino del Branch
62 assign pc_branch = idex_pc4_branch + (sign_ext_val << 2);
63 // Cálculo de la dirección del Jump
64 assign pc_jump = {ifid_pc4[31:28], jump_target, 2'b00};
65 // Mux del PC
66 assign pc_siguiente = (reset) ? 32'b0 : (jump_sig == 1'b1) ? pc_jump : (pcsource == 1'b1) ? pc_branch : pc_mas_4;
67 // Compuerta AND para el Branch (lógica de inversión del BNE ya está en DPTR)
68 assign pcsource = (!reset) ? (branch_sig & zero_sig) : 1'b0;
69
70 endmodule

```

Módulo MIPS_TM en editor de texto.

Módulo ALU_Control

```

module ALUControl (
    input [5:0] fnc,
    input [2:0] ALUOp,
    output reg [3:0] Sel
);
    always @(*) begin
        Sel = 4'b1111;
        case (ALUOp)
            // tipo i
            3'b000: Sel = 4'b0010; // Fuerza SUMA (lw, sw, addi)
            3'b001: Sel = 4'b0110; // Fuerza RESTA (beq, subi)
            3'b110: Sel = 4'b0110; // Fuerza RESTA para el BNE
            3'b011: Sel = 4'b0000; // Fuerza AND (andi)
            3'b100: Sel = 4'b0001; // Fuerza OR (ori)
            3'b101: Sel = 4'b0111; // Fuerza SLT (slti)

            // tipo r
            3'b010: begin
                case (fnc)
                    6'b100000: Sel = 4'b0010; // add
                    6'b100010: Sel = 4'b0110; // sub
                    6'b100100: Sel = 4'b0000; // and
                    6'b100101: Sel = 4'b0001; // or
                    6'b101010: Sel = 4'b0111; // slt
                    6'b100110: Sel = 4'b0011; // xor
                    6'b000000: Sel = 4'b1111; // nop
                    6'b000011: Sel = 4'b1000; // sra
                    6'b000010: Sel = 4'b1001; // srl
                endcase
            end
        endcase
    end
endmodule

```

```

        6'b110100: Sel = 4'b1100; // teq
        6'b110000: Sel = 4'b1101; // tge
        default: Sel = 4'b0000;
    endcase
end

    default: Sel = 4'b0000;
endcase
end
endmodule

```

```

1 module ALUControl (
2     input [5:0] fnc,
3     input [2:0] ALUOp,
4     output reg [3:0] Sel
5 );
6     always @(*) begin
7         Sel = 4'b1111;
8         case (ALUOp)
9             // tipo i
10            3'b000: Sel = 4'b0010; // Fuerza SUMA (lw, sw, addi)
11            3'b001: Sel = 4'b0110; // Fuerza RESTA (beq, subi)
12            3'b110: Sel = 4'b0110; // Fuerza RESTA para el BNE <--- NUEVO
13            3'b011: Sel = 4'b0000; // Fuerza AND (andi)
14            3'b100: Sel = 4'b0001; // Fuerza OR (ori)
15            3'b101: Sel = 4'b0111; // Fuerza SLT (slti)
16
17            // tipo r
18            3'b010: begin
19                case (fnc)
20                    6'b100000: Sel = 4'b0010; // add
21                    6'b100010: Sel = 4'b0110; // sub
22                    6'b100100: Sel = 4'b0000; // and
23                    6'b100101: Sel = 4'b0001; // or
24                    6'b101010: Sel = 4'b0111; // slt
25                    6'b100110: Sel = 4'b0011; // xor
26                    6'b000000: Sel = 4'b1111; // nop
27                    6'b000011: Sel = 4'b1000; // sra
28                    6'b000010: Sel = 4'b1001; // srl
29                    6'b110100: Sel = 4'b1100; // teq
30                    6'b110000: Sel = 4'b1101; // tge
31                    default: Sel = 4'b0000;
32                endcase
33            end
34
35            default: Sel = 4'b0000;
36        endcase
37    end
38 endmodule

```

Módulo ALU_Control en editor de texto.

Módulo ALU

```

module ALU (
    input [31:0] A, B,
    input [4:0] shamt, //para sra y srl
    input [3:0] ALUCtrl,
    output reg [31:0] Result,

```

```

        output reg Exception, //senal de teq y tge
        output reg Zero
    );
    always @(*) begin
        Result = 0;
        Exception = 0;
        Zero = 0;
        case (ALUCtrl)
            4'b0010: Result = A + B;      // add
            4'b0110: Result = A - B;      // sub
            4'b0000: Result = A & B;      // and
            4'b0001: Result = A | B;      //or
            4'b0111: Result = ($signed(A) < $signed(B)) ? 1:0; // slt
            4'b0011: Result = A ^ B;      // xor
            4'b1000: Result = $signed(B) >>> shamt; // sra
            4'b1001: Result = B >> shamt; // srl
            4'b1100: Exception = (A == B); // teq
            4'b1101: Exception = (A >= B); // tge
            4'b1111: Result = 0;          // nop

            default: begin
                Result = 0;
                Exception = 0;
            end
        endcase
        // Señal Zero
        Zero = (Result == 0) ? 1'b1 : 1'b0;
    end
endmodule

```

```

1 module ALU (
2     input [31:0] A, B,
3     input [4:0] shamt, //para sra y srl
4     input [3:0] ALUCtrl,
5     output reg [31:0] Result,
6     output reg Exception, //senal de teq y tge
7     output reg Zero
8 );
9
10 always @(*) begin
11     Result = 0;
12     Exception = 0;
13     Zero = 0;
14     case (ALUCtrl)
15         4'b0010: Result = A + B; // add
16         4'b0110: Result = A - B; // sub
17         4'b0000: Result = A & B; // and
18         4'b0001: Result = A | B; //or
19         4'b0111: Result = ($signed(A) < $signed(B)) ? 1:0; // slt
20         4'b0011: Result = A ^ B; // xor
21         4'b1000: Result = $signed(B) >>> shamt; // sra
22         4'b1001: Result = B >> shamt; // srl
23         4'b1100: Exception = (A == B); // teq
24         4'b1101: Exception = (A >= B); // tge
25         4'b1111: Result = 0; // nop
26
27     default: begin
28         Result = 0;
29         Exception = 0;
30     end
31 endcase
32 // Señal Zero
33 Zero = (Result == 0) ? 1'b1 : 1'b0;
34 end
endmodule

```

Módulo ALU en editor de texto.

Módulo Banco de Registros

```

module BancoReg (
    input clk,
    input RegWrite,
    input [4:0] rs, rt, rd,
    input [31:0] writeData,
    output [31:0] readData1, readData2
);

reg [31:0] regs [0:31];

//inicializar para evitar el error
integer i;
initial begin
    for (i = 0; i < 32; i = i + 1) begin
        regs[i] = 32'b0;
    end
end

assign readData1 = (RegWrite && (rd != 0) && (rd == rs)) ? writeData : regs[rs];
assign readData2 = (RegWrite && (rd != 0) && (rd == rt)) ? writeData : regs[rt];

always @(posedge clk) begin
    if (RegWrite && rd != 0)

```

```

    regs[rd] <= writeData;
end

endmodule

```

```

1  module BancoReg (
2      input clk,
3      input RegWrite,
4      input [4:0] rs, rt, rd,
5      input [31:0] writeData,
6      output [31:0] readData1, readData2
7  );
8
9      reg [31:0] regs [0:31];
10
11     //inicializar para evitar el error
12     integer i;
13     initial begin
14         for (i = 0; i < 32; i = i + 1) begin
15             ...
16             regs[i] = 32'b0;
17         end
18     end
19
20     assign readData1 = (RegWrite && (rd != 0) && (rd == rs)) ? writeData : regs[rs];
21     assign readData2 = (RegWrite && (rd != 0) && (rd == rt)) ? writeData : regs[rt];
22
23     always @(posedge clk) begin
24         if (RegWrite && rd != 0)
25             regs[rd] <= writeData;
26     end
27 endmodule

```

Módulo BancoReg en editor de texto.

Módulo DPTR

```

module DPTR (
    input clk,
    input reset,
    input [31:0] instruction,
    input [31:0] pc4_in,      //Recibe PC+4 de la etapa ID
    output [31:0] result,

    output Branch_out,
    output Zero_out,
    output [31:0] SignExt_out,
    output [31:0] PC4_Branch_out, //Exporta PC+4 desde la etapa EX

    output Jump_out,
    output [25:0] JumpTarget_out
);

    wire [5:0] op = instruction[31:26];
    wire [4:0] rs = instruction[25:21];
    wire [4:0] rt = instruction[20:16];

```

```

wire [4:0] rd = instruction[15:11];
wire [4:0] sh = instruction[10:6];
wire [5:0] funct = instruction[5:0];
wire [15:0] inmed = instruction[15:0];

wire [5:0] idex_funct;
wire [4:0] idex_shamt;
wire [4:0] idex_rt;
wire [4:0] idex_rd;
wire [4:0] idex_rs;
wire idex_RegDst;
wire [4:0] exmem_writeReg;
wire [4:0] memwb_writeReg;

wire RegDst, ALUSrc, Branch;
wire memToReg, RegWrite, memToWrite, memToRead;
wire [2:0] ALUOp;
wire [2:0] idex_ALUOp;
wire [3:0] ALUCtrl;

wire aluException;
wire [31:0] readData1, readData2, aluResult;
wire [31:0] sign_ext_out;
wire [31:0] idex_rd1;
wire [31:0] idex_rd2;
wire [31:0] idex_imm;
wire [31:0] idex_pc4; // CABLE PARA EL PC DEL BUFFER ID/EX
wire idex_ALUSrc;
wire [31:0] exmem_aluResult;
wire [31:0] exmem_writeData;

wire exmem_Zero;
wire exmem_Branch;
wire idex_Branch;
wire aluZero;

wire exmem_RegWrite;
wire exmem_MemWrite;
wire exmem_MemToReg;
wire idex_RegWrite;
wire idex_MemWrite;
wire idex_MemToReg;
wire [31:0] memwb_memData;
wire [31:0] memwb_aluResult;

wire memwb_RegWrite;
wire memwb_MemToReg;
wire [4:0] write_reg_addr;
wire [31:0] alu_b_in;
wire [31:0] mem_data_out;
wire [31:0] write_data_in;

```

```
wire Jump;
```

```
wire [1:0] ForwardA, ForwardB;
```

```
wire [31:0] alu_operand_A, alu_operand_B_pre_mux;
```

```
U_Control UC(  
    .op(op),  
    .funct(funct),  
    .RegDst(RegDst),  
    .ALUSrc(ALUSrc),  
    .Branch(Branch),  
    .memToReg(memToReg),  
    .memToWrite(memToWrite),  
    .memToRead(memToRead),  
    .AluOP(ALUOp),  
    .RegWrite(RegWrite),  
    .Jump(Jump),  
    .Bne()  
);
```

```
ALUControl ALUC(  
    .fnc(idex_funct),  
    .ALUOp(idex_ALUOp),  
    .Sel(ALUCtrl)  
);
```

```
Mux2_1_5 mux_regdst(  
    .in0(idex_rt),  
    .in1(idex_rd),  
    .sel(idex_RegDst),  
    .out(write_reg_addr)  
);
```

```
SignExtend se(  
    .in(inmed),  
    .out(sign_ext_out)  
);
```

```
BancoReg BR (  
    .clk(clk),  
    .RegWrite(memwb_RegWrite),  
    .rs(rs),  
    .rt(rt),  
    .rd(memwb_writeReg),  
    .writeData(write_data_in),  
    .readData1(readData1),  
    .readData2(readData2)  
);
```

```
assign alu_operand_A = (ForwardA == 2'b00) ? idex_rd1 :  
    (ForwardA == 2'b10) ? exmem_aluResult :  
    (ForwardA == 2'b01) ? write_data_in :
```

```

        idex_rd1;

assign alu_operand_B_pre_mux = (ForwardB == 2'b00) ? idex_rd2 :
    (ForwardB == 2'b10) ? exmem_aluResult :
    (ForwardB == 2'b01) ? write_data_in :
        idex_rd2;

wire [31:0] final_srcB = idex_ALUSrc ? idex_imm : alu_operand_B_pre_mux;

ALU alu(
    .A(alu_operand_A),
    .B(final_srcB),
    .shamt(idex_shamt),
    .ALUCtrl(ALUCtrl),
    .Result(aluResult),
    .Exception(aluException),
    .Zero(aluZero)
);

BufferDatos idex(
    .clk(clk),
    .reset(reset),

    .pc4_in(pc4_in),    // RECIBE DE LA ETAPA ID
    .pc4_out(idex_pc4), // GUARDA PARA LA ETAPA EX

    .rd1_in(readData1),
    .rd2_in(readData2),
    .imm_in(sign_ext_out),

    .rs_in(rs),
    .rt_in(rt),
    .rd_in(rd),

    .RegWrite_in(RegWrite),
    .MemToReg_in(memToReg),
    .MemWrite_in(memToWrite),
    .MemRead_in(memToRead),
    .ALUSrc_in(ALUSrc),
    .RegDst_in(RegDst),
    .Branch_in(Branch),
    .ALUOp_in(ALUOp),

    .funct_in(funct),
    .shamt_in(sh),
    .shamt_out(idex_shamt),
    .funct_out(idex_funct),

    .rd1_out(idex_rd1),
    .rd2_out(idex_rd2),
    .imm_out(idex_imm),

```

```

        .rs_out(idex_rs),
        .rt_out(idex_rt),
        .rd_out(idex_rd),

        .RegWrite_out(idex_RegWrite),
        .MemWrite_out(idex_MemWrite),
        .MemToReg_out(idex_MemToReg),
        .MemRead_out(),
        .ALUSrc_out(idex_ALUSrc),
        .RegDst_out(idex_RegDst),
        .Branch_out(idex_Branch),
        .ALUOp_out(idex_ALUOp)
    );

    BufferALU exmem(
        .clk(clk),
        .reset(reset),

        .aluResult_in(aluResult),
        .writeData_in(alu_operand_B_pre_mux),
        .writeReg_in(write_reg_addr),

        .Zero_in(aluZero),
        .Branch_in(idex_Branch),

        .aluResult_out(exmem_aluResult),
        .writeData_out(exmem_writeData),
        .writeReg_out(exmem_writeReg),

        .Zero_out(exmem_Zero),
        .Branch_out(exmem_Branch),

        .RegWrite_in(idex_RegWrite),
        .MemToReg_in(idex_MemToReg),
        .MemWrite_in(idex_MemWrite),
        .MemRead_in(1'b0),

        .RegWrite_out(exmem_RegWrite),
        .MemToReg_out(exmem_MemToReg),
        .MemWrite_out(exmem_MemWrite),
        .MemRead_out()
    );

    Mem_D dmem(
        .clk(clk),
        .MemWrite(exmem_MemWrite),
        .dir(exmem_aluResult),
        .DEntrada(exmem_writeData),
        .DSalida(mem_data_out)
    );

    BufferFinal memwb(

```

```

        .clk(clk),
        .reset(reset),
        .memData_in(mem_data_out),
        .aluResult_in(exmem_aluResult),

        .writeReg_in(exmem_writeReg),

        .RegWrite_in(exmem_RegWrite),
        .MemToReg_in(exmem_MemToReg),

        .memData_out(memwb_memData),
        .aluResult_out(memwb_aluResult),

        .writeReg_out(memwb_writeReg),

        .RegWrite_out(memwb_RegWrite),
        .MemToReg_out(memwb_MemToReg)
    );

    Mux2_1_32 mux_memtoreg(
        .in0(memwb_aluResult),
        .in1(memwb_memData),
        .sel(memwb_MemToReg),
        .out(write_data_in)
    );

    Forwarding_Unit FU (
        .idex_rs(idex_rs),
        .idex_rt(idex_rt),
        .exmem_rd(exmem_writeReg),
        .memwb_rd(memwb_writeReg),
        .exmem_RegWrite(exmem_RegWrite),
        .memwb_RegWrite(memwb_RegWrite),
        .ForwardA(ForwardA),
        .ForwardB(ForwardB)
    );

    // Identificador para invertir el comportamiento del BNE
    wire idex_is_bne = (idex_ALUOp == 3'b110);

    //Evaluar el salto en la etapa EX
    assign result = aluResult;
    assign Branch_out = idex_Branch;

    // Si es BNE, el salto se da cuando Zero es 0. Si es BEQ, cuando es 1.
    assign Zero_out = idex_is_bne ? ~aluZero : aluZero;

    assign SignExt_out = idex_imm;
    assign PC4_Branch_out = idex_pc4; // Mandamos el PC correcto al MIPS_TM;
    assign Jump_out = Jump;
    assign JumpTarget_out = instruction[25:0];
endmodule

```

```

MIPS_TM.v Banco_Reg.v DPTR.v U_Control.v Buffer.v TestF3_MemInst.mem
1 module DPTR (
2     input clk,
3     input reset,
4     input [31:0] instruction,
5     input [31:0] pc4_in,          //Recibe PC+4 de la etapa ID
6     output [31:0] result,
7
8     output Branch_out,
9     output Zero_out,
10    output [31:0] SignExt_out,
11    output [31:0] PC4_Branch_out, //Exporta PC+4 desde la etapa EX
12
13    output Jump_out,
14    output [25:0] JumpTarget_out
15 );
16
17    wire [5:0] op = instruction[31:26];
18    wire [4:0] rs = instruction[25:21];
19    wire [4:0] rt = instruction[20:16];
20    wire [4:0] rd = instruction[15:11];
21    wire [4:0] sh = instruction[10:6];
22    wire [5:0] funct = instruction[5:0];
23    wire [15:0] immed = instruction[15:0];
24
25    wire [5:0] idex_funct;
26    wire [4:0] idex_shamt;
27    wire [4:0] idex_rt;
28    wire [4:0] idex_rd;
29    wire [4:0] idex_rs;
30    wire idex_RegDst;
31    wire [4:0] exmem_writeReg;
32    wire [4:0] memwb_writeReg;
33
34    wire RegDst, ALUSrc, Branch;
35    wire memToReg, RegWrite, memToWrite, memToRead;
36    wire [2:0] ALUOp;
37    wire [2:0] idex_ALUOp;
38    wire [3:0] ALUCtrl;
39
40    wire aluException;
41    wire [31:0] readData1, readData2, aluResult;
42    wire [31:0] sign_ext_out;
43
44    wire [31:0] idex_rdl;
45    wire [31:0] idex_rd2;
46    wire [31:0] idex_imm;
47    wire [31:0] idex_pc4; // CABLE PARA EL PC DEL BUFFER ID/EX
48    wire idex_ALUSrc;
49    wire [31:0] exmem_aluResult;
50    wire [31:0] exmem_writeData;
51
52    wire exmem_Zero;
53    wire exmem_Branch;
54    wire idex_Branch;
55    wire aluZero;
56
57    wire exmem_RegWrite;
58    wire exmem_MemWrite;
59    wire exmem_MemToReg;
60    wire idex_RegWrite;
61    wire idex_MemWrite;
62    wire idex_MemToReg;
63    wire [31:0] memwb_memData;
64    wire [31:0] memwb_aluResult;
65
66    wire memwb_RegWrite;
67    wire memwb_MemToReg;
68    wire [4:0] write_reg_addr;
69    wire [31:0] alu_b_in;
70    wire [31:0] mem_data_out;
71    wire [31:0] write_data_in;
72
73    wire Jump;
74
75    wire [1:0] ForwardA, ForwardB;
76    wire [31:0] alu_operand_A, alu_operand_B_pre_mux;
77
78    U_Control UC(
79        .op(op),
80        .funct(funct),
81        .RegDst(RegDst),
82        .ALUSrc(ALUSrc),

```

```

82     .Branch(Branch),
83     .memToReg(memToReg),
84     .memToWrite(memToWrite),
85     .memToRead(memToRead),
86     .AluOp(ALUOp),
87     .RegWrite(RegWrite),
88     .Jump(Jump),
89     .Bne()
90 );
91
92 ALUControl ALUC(
93     .fnc(idex_funct),
94     .ALUOp(idex_ALUOp),
95     .Sel(ALUCtrl)
96 );
97
98 Mux2_1_5 mux_regdst(
99     .in0(idex_rt),
100     .in1(idex_rd),
101     .sel(idex_RegDst),
102     .out(write_reg_addr)
103 );
104
105 SignExtend se(
106     .in(immed),
107     .out(sign_ext_out)
108 );
109
110 BancoReg BR (
111     .clk(clk),
112     .RegWrite(memwb_RegWrite),
113     .rs(rs),
114     .rt(rt),
115     .rd(memwb_writeReg),
116     .writeData(write_data_in),
117     .readData1(readData1),
118     .readData2(readData2)
119 );
120
121 assign alu_operand_A = (ForwardA == 2'b00) ? idex_rd1 :
122                        (ForwardA == 2'b10) ? exmem_aluResult :
123                        (ForwardA == 2'b01) ? write_data_in :
124                        idex_rd1;
125
126 assign alu_operand_B_pre_mux = (ForwardB == 2'b00) ? idex_rd2 :
127                                (ForwardB == 2'b10) ? exmem_aluResult :
128                                (ForwardB == 2'b01) ? write_data_in :
129                                idex_rd2;
130
131 wire [31:0] final_srcB = idex_ALUSrc ? idex_imm : alu_operand_B_pre_mux;
132
133 ALU alu(
134     .A(alu_operand_A),
135     .B(final_srcB),
136     .shamt(idex_shamt),
137     .ALUCtrl(ALUCtrl),
138     .Result(aluResult),
139     .Exception(aluException),
140     .Zero(aluZero)
141 );
142
143 BufferDatos idex(
144     .clk(clk),
145     .reset(reset),
146
147     .pc4_in(pc4_in),      // RECIBE DE LA ETAPA ID
148     .pc4_out(idex_pc4),  // GUARDA PARA LA ETAPA EX
149
150     .rd1_in(readData1),
151     .rd2_in(readData2),
152     .imm_in(sign_ext_out),
153
154     .rs_in(rs),
155     .rt_in(rt),
156     .rd_in(rd),
157
158     .RegWrite_in(RegWrite),
159     .MemToReg_in(memToReg),
160     .MemWrite_in(memToWrite),
161     .MemRead_in(memToRead),
162     .ALUSrc_in(ALUSrc),
163     .RegDst_in(RegDst),
164     .Branch_in(Branch),

```

```

165     .ALUOp_in(ALUOp),
166
167     .funct_in(funct),
168     .shamt_in(sh),
169     .shamt_out(idex_shamt),
170     .funct_out(idex_funct),
171
172     .rd1_out(idex_rd1),
173     .rd2_out(idex_rd2),
174     .imm_out(idex_imm),
175
176     .rs_out(idex_rs),
177     .rt_out(idex_rt),
178     .rd_out(idex_rd),
179
180     .RegWrite_out(idex_RegWrite),
181     .MemWrite_out(idex_MemWrite),
182     .MemToReg_out(idex_MemToReg),
183     .MemRead_out(),
184     .ALUSrc_out(idex_ALUSrc),
185     .RegDst_out(idex_RegDst),
186     .Branch_out(idex_Branch),
187     .ALUOp_out(idex_ALUOp)
188 );
189
190 BufferALU exmem(
191     .clk(clk),
192     .reset(reset),
193
194     .aluResult_in(aluResult),
195     .writeData_in(alu_operand_B_pre_mux),
196     .writeReg_in(write_reg_addr),
197
198     .Zero_in(aluZero),
199     .Branch_in(idex_Branch),
200
201     .aluResult_out(exmem_aluResult),
202     .writeData_out(exmem_writeData),
203     .writeReg_out(exmem_writeReg),
204
205     .Zero_out(exmem_Zero),
206     .Branch_out(exmem_Branch),
207
208     .RegWrite_in(idex_RegWrite),
209     .MemToReg_in(idex_MemToReg),
210     .MemWrite_in(idex_MemWrite),
211     .MemRead_in(1'b0),
212
213     .RegWrite_out(exmem_RegWrite),
214     .MemToReg_out(exmem_MemToReg),
215     .MemWrite_out(exmem_MemWrite),
216     .MemRead_out()
217 );
218
219 Mem_D dmem(
220     .clk(clk),
221     .MemWrite(exmem_MemWrite),
222     .dir(exmem_aluResult),
223     .DEntrada(exmem_writeData),
224     .DSalida(mem_data_out)
225 );
226
227 BufferFinal memwb(
228     .clk(clk),
229     .reset(reset),
230     .memData_in(mem_data_out),
231     .aluResult_in(exmem_aluResult),
232
233     .writeReg_in(exmem_writeReg),
234
235     .RegWrite_in(exmem_RegWrite),
236     .MemToReg_in(exmem_MemToReg),
237
238     .memData_out(memwb_memData),
239     .aluResult_out(memwb_aluResult),
240
241     .writeReg_out(memwb_writeReg),
242
243     .RegWrite_out(memwb_RegWrite),
244     .MemToReg_out(memwb_MemToReg)
245 );

```

```

247     Mux2_1_32 mux_memtoreg(
248         .in0(memwb_aluResult),
249         .in1(memwb_memData),
250         .sel(memwb_MemToReg),
251         .out(write_data_in)
252     );
253
254     Forwarding_Unit FU (
255         .idex_rs(idex_rs),
256         .idex_rt(idex_rt),
257         .exmem_rd(exmem_writeReg),
258         .memwb_rd(memwb_writeReg),
259         .exmem_RegWrite(exmem_RegWrite),
260         .memwb_RegWrite(memwb_RegWrite),
261         .ForwardA(ForwardA),
262         .ForwardB(ForwardB)
263     );
264
265     // Identificador para invertir el comportamiento del BNE
266     wire idex_is_bne = (idex_ALUOp == 3'b110);
267
268     //Evaluar el salto en la etapa EX
269     assign result = aluResult;
270     assign Branch_out = idex_Branch;
271
272     // Si es BNE, el salto se da cuando Zero es 0. Si es BEQ, cuando es 1.
273     assign Zero_out = idex_is_bne ? ~aluZero : aluZero;
274
275     assign SignExt_out = idex_imm;
276     assign PC4_Branch_out = idex_pc4; // Mandamos el PC correcto al MIPS_TM
277
278     assign Jump_out = Jump;
279     assign JumpTarget_out = instruction[25:0];
280 endmodule

```

Módulo DPTR en editor de texto.

Módulo Memoria de Datos

```

module Mem_D(
    input clk,
    input MemWrite,    //1 para escribir, 0 para leer
    input [31:0] dir,
    input [31:0] DEntrada,
    output reg [31:0] DSalida
);

    reg [31:0] MEM [0:255];

    integer i;

    initial begin
        for(i = 0; i < 256; i = i + 1)
            MEM[i] = 32'b0;
        end

        //divide dir entre 4
        always @(*) begin
            if (^dir === 1'bx)
                DSalida = 32'b0;
            else
                DSalida = MEM[(dir >> 2) & 8'hFF];
        end
    end

```

```

// divide la dirección entre 4
always @(posedge clk)
begin
    if (MemWrite) begin
        MEM[(dir >> 2) & 8'hFF] <= DEntrada;
    end
end
endmodule

```

```

1  module Mem_D(
2      input clk,
3      input MemWrite,          //1 para escribir, 0 para leer
4      input [31:0] dir,
5      input [31:0] DEntrada,
6      output reg [31:0] DSalida
7  );
8
9      reg [31:0] MEM [0:255];
10
11     integer i;
12
13     initial begin
14         for(i = 0; i < 256; i = i + 1)
15             MEM[i] = 32'b0;
16         end
17
18     //divide dir entre 4
19     always @(*) begin
20         if (^dir == 1'bx)
21             DSalida = 32'b0;
22         else
23             DSalida = MEM[(dir >> 2) & 8'hFF];
24         end
25
26     // divide la dirección entre 4
27     always @(posedge clk)
28     begin
29         if (MemWrite) begin
30             MEM[(dir >> 2) & 8'hFF] <= DEntrada;
31         end
32     end
33 endmodule

```

Módulo MEM_D en ModelSim.

Módulo Mem I

```

module Mem_I(
    input [31:0] dir,
    output reg [31:0] DSalida
);

reg [7:0] MEM[0:1023];

integer i;

initial begin
    for(i=0;i<1024;i=i+1)
        MEM[i]=0;
        $readmemb("TestF3_MemInst.mem", MEM); //DIOS.txt

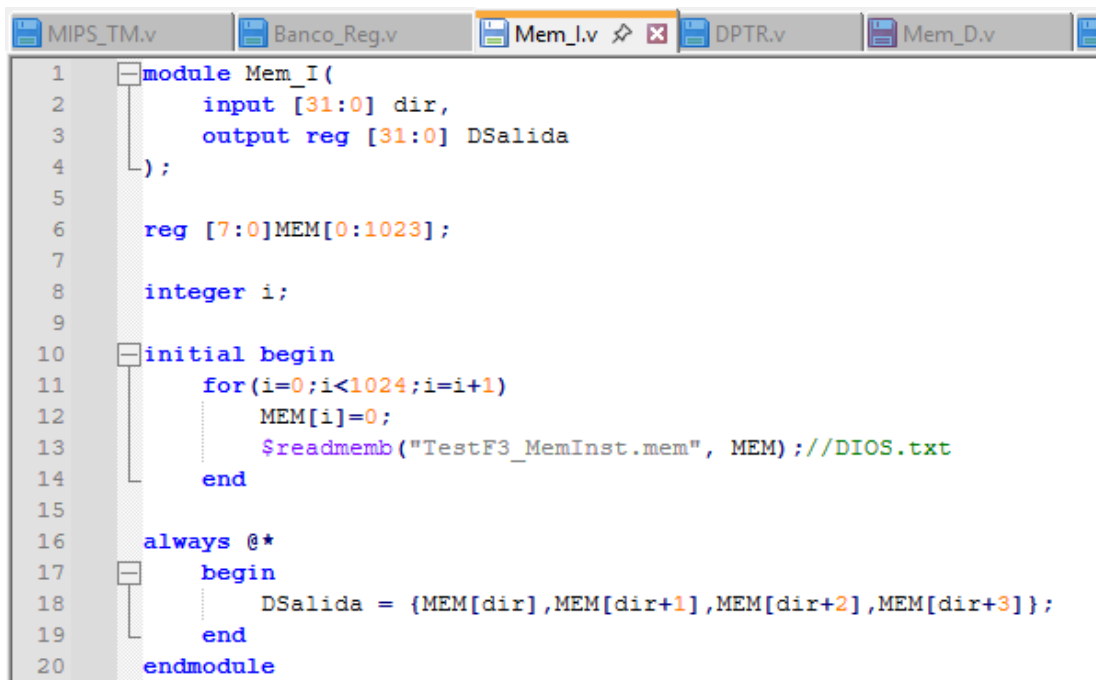
```

```

        end

always @*
    begin
        DSalida = {MEM[dir],MEM[dir+1],MEM[dir+2],MEM[dir+3]};
    end
endmodule

```



```

1  module Mem_I (
2      input [31:0] dir,
3      output reg [31:0] DSalida
4  );
5
6      reg [7:0] MEM[0:1023];
7
8      integer i;
9
10     initial begin
11         for(i=0;i<1024;i=i+1)
12             MEM[i]=0;
13         $readmemb("TestF3_MemInst.mem", MEM); //DIOS.txt
14     end
15
16     always @*
17     begin
18         DSalida = {MEM[dir],MEM[dir+1],MEM[dir+2],MEM[dir+3]};
19     end
20 endmodule

```

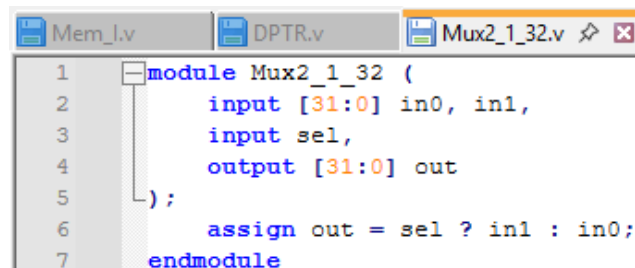
Módulo MEM_I en ModelSim.

Multiplexor 2 a 1 32 bits

```

module Mux2_1_32 (
    input [31:0] in0, in1,
    input sel,
    output [31:0] out
);
    assign out = sel ? in1 : in0;
endmodule

```



```

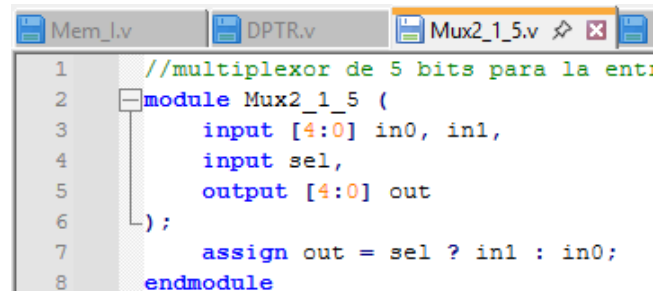
1  module Mux2_1_32 (
2      input [31:0] in0, in1,
3      input sel,
4      output [31:0] out
5  );
6      assign out = sel ? in1 : in0;
7  endmodule

```

Módulo Mux2_1_32 en ModelSim.

Multiplexor 2 a 1 5 bits

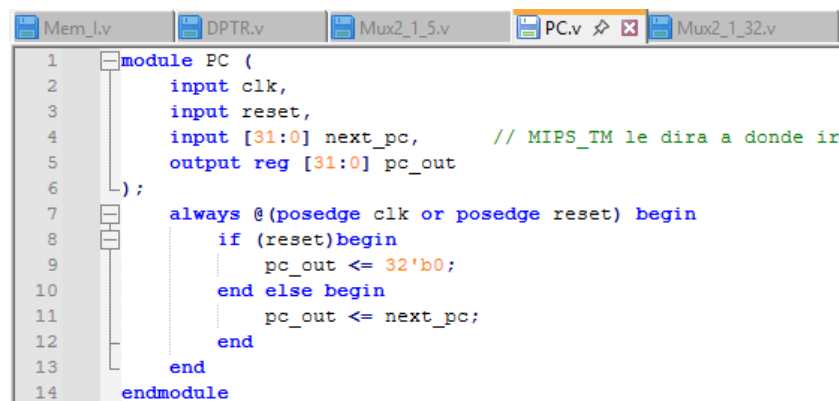
```
//multiplexor de 5 bits para la entrada del banco de registros write register
module Mux2_1_5 (
    input [4:0] in0, in1,
    input sel,
    output [4:0] out
);
    assign out = sel ? in1 : in0;
endmodule
```



Módulo Mux2_1_5 en ModelSim.

Módulo Program Counter

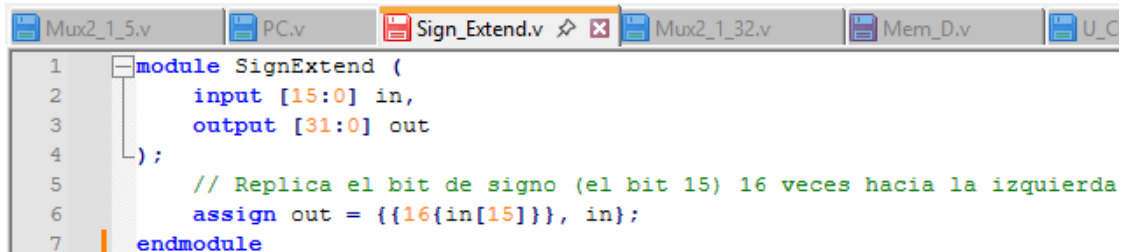
```
module PC (
    input clk,
    input reset,
    input [31:0] next_pc,    // MIPS_TM le dira a donde ir
    output reg [31:0] pc_out
);
    always @(posedge clk or posedge reset) begin
        if (reset)begin
            pc_out <= 32'b0;
        end else begin
            pc_out <= next_pc;
        end
    end
endmodule
```



Módulo PC en ModelSim.

Módulo Sign Extend

```
module SignExtend (  
    input [15:0] in,  
    output [31:0] out  
);  
    // Replica el bit de signo (el bit 15) 16 veces hacia la izquierda  
    assign out = {{16{in[15]}}, in};  
endmodule
```



Módulo SignExtend en ModelSim.

Módulo Test Bench DPTR

```
`timescale 1ns / 1ps  
module TB_DPTR;  
    reg clk;  
    reg reset;  
    wire [31:0] result;  
  
    MIPS_TM tm(clk, reset, result);  
  
    //10 unidades de tiempo para reloj  
    initial begin  
        clk = 0;  
        forever #5 clk = ~clk;  
    end  
  
    always @(posedge clk) begin//somo para monitorear  
        if(!reset) begin  
            $display(  
                "PC=%d Instr=%h RegWrite=%b writeReg=%d writeData=%d",  
                tm.pc_actual,  
                tm.ifid_instr,  
                tm.dp.memwb_RegWrite,  
                tm.dp.memwb_writeReg,  
                tm.dp.write_data_in  
            );  
        end  
    end  
  
    always @(posedge clk) begin//monitor x2
```

```

        $display(
        "ALUResult=%d MemData=%d dir=%d",
        tm.dp.exmem_aluResult,
        tm.dp.mem_data_out,
        tm.dp.exmem_aluResult >> 2
        );

    end

initial begin
    reset = 1;
    #10;
    //para cargar la memoria de datos con arreglo desordenado
        //utilizamos 45 en la busqueda binaria, para caso de exito hay que agregarlo
    tm.dp.dmem.MEM[0] = 32'd80;
    tm.dp.dmem.MEM[1] = 32'd20;
    tm.dp.dmem.MEM[2] = 32'd60;
    tm.dp.dmem.MEM[3] = 32'd40;
    tm.dp.dmem.MEM[4] = 32'd100;
    tm.dp.dmem.MEM[5] = 32'd5;
    tm.dp.dmem.MEM[6] = 32'd15;
    tm.dp.dmem.MEM[7] = 32'd50;
    tm.dp.dmem.MEM[8] = 32'd25;
    tm.dp.dmem.MEM[9] = 32'd30; // (El arreglo tiene n=10)

    tm.dp.BR.regs[0] = 32'd0;

    #10;
    reset = 0; //activa el program counter

    $display("Ejecutando Bubble sort y Busqueda Binaria...");

    // necesitamos darle suficiente tiempo para que complete los bucles.
    #200000;

    $display("Simulacion finalizada. Revisa el registro $v0 (regs[2]) para el resultado.");
    $stop;
end
endmodule

```

```

1  `timescale 1ns / 1ps
2  module TB_DPTR;
3      reg clk;
4      reg reset;
5      wire [31:0] result;
6
7      MIPS_TM tm(clk, reset, result);
8
9      //10 unidades de tiempo para reloj
10     initial begin
11         clk = 0;
12         forever #5 clk = ~clk;
13     end
14
15     always @(posedge clk) begin//somo para monitorear
16         if(!reset) begin
17             $display(
18                 "PC=%d Instr=%h RegWrite=%b writeReg=%d writeData=%d",
19                 tm.pc_actual,
20                 tm.ifid_instr,
21                 tm.dp.memwb_RegWrite,
22                 tm.dp.memwb_writeReg,
23                 tm.dp.write_data_in
24             );
25         end
26     end
27
28     always @(posedge clk) begin//monitor x2
29
30         $display(
31             "ALUResult=%d MemData=%d dir=%d",
32             tm.dp.exmem_aluResult,
33             tm.dp.mem_data_out,
34             tm.dp.exmem_aluResult >> 2
35         );

```

```

37     end
38
39     initial begin
40         reset = 1;
41         #10;
42         //para cargar la memodria de datos con arreglo desordenado
43         //utilizamos 45 en la busqueda binaria, para caso de exito hay que agregarlo
44         tm.dp.dmem.MEM[0] = 32'd80;
45         tm.dp.dmem.MEM[1] = 32'd20;
46         tm.dp.dmem.MEM[2] = 32'd60;
47         tm.dp.dmem.MEM[3] = 32'd40;
48         tm.dp.dmem.MEM[4] = 32'd100;
49         tm.dp.dmem.MEM[5] = 32'd5;
50         tm.dp.dmem.MEM[6] = 32'd15;
51         tm.dp.dmem.MEM[7] = 32'd50;
52         tm.dp.dmem.MEM[8] = 32'd25;
53         tm.dp.dmem.MEM[9] = 32'd30; // (El arreglo tiene n=10)
54
55         tm.dp.BR.reg[0] = 32'd0;
56
57         #10;
58         reset = 0; //activa el program counter
59
60         $display("Ejecutando Bubble sort y Busqueda Binaria...");
61
62         // necesitamos darle suficiente tiempo para que complete los bucles.
63         #200000;
64
65         $display("Simulacion finalizada. Revisa el registro $v0 (regs[2]) para el resultado.");
66         $stop;
67     end
68 endmodule

```

Módulo TB_DPTR en ModelSim.

Módulo Unidad de Control

```
module U_Control(  
    input [5:0] op,  
    input [5:0] funct,  
    // Señales fase 2  
    output reg RegDst,  
    output reg ALUSrc,  
    output reg Branch,  
    // fase 1  
    output reg memToReg,  
    output reg memToWrite,  
    output reg memToRead,  
    output reg [2:0] AluOP,  
    output reg RegWrite,  
    output reg Jump,  
    output reg Bne  
);  
  
always@(*) begin  
    // Valores por defecto  
    RegDst = 1'b0; ALUSrc = 1'b0; Branch = 1'b0;  
    RegWrite = 1'b0; AluOP = 3'b000; memToRead = 1'b0;  
    memToWrite = 1'b0; memToReg = 1'b0; Jump = 1'b0; Bne = 1'b0;  
  
    case (op)  
        // TIPO R  
        6'b000000: begin  
            RegDst = 1'b1; // Destino es rd  
            ALUSrc = 1'b0;  
            AluOP = 3'b010; // Código para ir al case(funct)  
  
            if(funct == 6'b110100 || funct == 6'b110000) // para teq y tge  
                RegWrite = 1'b0;  
            else  
                RegWrite = 1'b1;  
        end  
  
        // LW (Load Word)  
        6'b100011: begin  
            RegDst = 1'b0;  
            ALUSrc = 1'b1;  
            memToReg = 1'b1; // Viene de Memoria  
            RegWrite = 1'b1;  
            memToRead = 1'b1;  
            AluOP = 3'b000; // Suma  
        end  
  
        // SW (Store Word)  
        6'b101011: begin  
            ALUSrc = 1'b1;  
            memToWrite = 1'b1; // Escribe en Memoria  
            AluOP = 3'b000; // Suma
```

end

// BEQ (Branch if Equal)

6'b000100: begin

 ALUSrc = 1'b0;

 Branch = 1'b1;

 Bne = 1'b0;

 AluOP = 3'b001;

end

// BNE

6'b000101: begin

 ALUSrc = 1'b0;

 Branch = 1'b1;

 Bne = 1'b1;

 AluOP = 3'b110; //especial para BNE

end

// ADDI

6'b001000: begin // opcode de ADDI

 RegDst = 1'b0;

 ALUSrc = 1'b1;

 RegWrite = 1'b1;

 AluOP = 3'b000; // SUMA

end

// ANDI

6'b001100: begin

 RegDst = 1'b0;

 ALUSrc = 1'b1;

 RegWrite = 1'b1;

 AluOP = 3'b011; // ALUControl hará AND

end

// ORI

6'b001101: begin

 RegDst = 1'b0;

 ALUSrc = 1'b1;

 RegWrite = 1'b1;

 AluOP = 3'b100; // ALUControl hará OR

end

// SLTI

6'b001010: begin

 RegDst = 1'b0;

 ALUSrc = 1'b1;

 RegWrite = 1'b1;

 AluOP = 3'b101;

end

// SUBI

6'b001001: begin

```

        RegDst = 1'b0;
        ALUSrc = 1'b1;
        RegWrite = 1'b1;
        AluOP = 3'b001; // ALUControl hará RESTA
    end

    //TIPO J
    6'b000010: begin
        Jump = 1'b1; // señal de salto incondicional
    end
endcase
end
endmodule

```

```

1  module U_Control(
2      input [5:0] op,
3      input [5:0] funct,
4      // Señales fase 2
5      output reg RegDst,
6      output reg ALUSrc,
7      output reg Branch,
8      // fase 1
9      output reg memToReg,
10     output reg memToWrite,
11     output reg memToRead,
12     output reg [2:0] AluOP,
13     output reg RegWrite,
14     output reg Jump,
15     output reg Bne
16 );
17
18 always@(*) begin
19     // Valores por defecto
20     RegDst = 1'b0; ALUSrc = 1'b0; Branch = 1'b0;
21     RegWrite = 1'b0; AluOP = 3'b000; memToRead = 1'b0;
22     memToWrite = 1'b0; memToReg = 1'b0; Jump = 1'b0; Bne = 1'b0;
23
24     case (op)
25         // TIPO R
26         6'b000000: begin
27             RegDst = 1'b1; // Destino es rd
28             ALUSrc = 1'b0;
29             AluOP = 3'b010; // Código para ir al case(funct)
30
31             if(funct == 6'b110100 || funct == 6'b110000) // para teq y tge
32                 RegWrite = 1'b0;
33             else
34                 RegWrite = 1'b1;
35         end

```

```

37 // LW (Load Word)
38 6'b100011: begin
39     RegDst = 1'b0;
40     ALUSrc = 1'b1;
41     memToReg = 1'b1; // Viene de Memoria
42     RegWrite = 1'b1;
43     memToRead = 1'b1;
44     AluOP = 3'b000; // Suma
45 end
46
47 // SW (Store Word)
48 6'b101011: begin
49     ALUSrc = 1'b1;
50     memToWrite = 1'b1; // Escribe en Memoria
51     AluOP = 3'b000; // Suma
52 end
53
54 // BEQ (Branch if Equal)
55 6'b000100: begin
56     ALUSrc = 1'b0;
57     Branch = 1'b1;
58     Bne = 1'b0;
59     AluOP = 3'b001;
60 end
61
62 // BNE
63 6'b000101: begin
64     ALUSrc = 1'b0;
65     Branch = 1'b1;
66     Bne = 1'b1;
67     AluOP = 3'b110; //especial para BNE
68 end

```

```

70 // ADDI
71 6'b001000: begin // opcode de ADDI
72     RegDst = 1'b0;
73     ALUSrc = 1'b1;
74     RegWrite = 1'b1;
75     AluOP = 3'b000; // SUMA
76 end
77
78 // ANDI
79 6'b001100: begin
80     RegDst = 1'b0;
81     ALUSrc = 1'b1;
82     RegWrite = 1'b1;
83     AluOP = 3'b011; // ALUControl hará AND
84 end
85
86 // ORI
87 6'b001101: begin
88     RegDst = 1'b0;
89     ALUSrc = 1'b1;
90     RegWrite = 1'b1;
91     AluOP = 3'b100; // ALUControl hará OR
92 end
93
94 // SLTI
95 6'b001010: begin
96     RegDst = 1'b0;
97     ALUSrc = 1'b1;
98     RegWrite = 1'b1;
99     AluOP = 3'b101;
100 end
101
102 // SUBI
103 6'b001001: begin
104     RegDst = 1'b0;

```

```

105     ALUSrc = 1'b1;
106     RegWrite = 1'b1;
107     ALuOP = 3'b001; // ALUControl hará RESTA
108 end
109
110 //TIPO J
111 6'b000010: begin
112     Jump = 1'b1; // señal de salto incondicional
113 end
114 endcase
115 end
116 endmodule

```

Módulo U_Control en ModelSim.

Módulo Forward Unit

```

//resolves data hazards (ej. Read-After-Write)
module Forwarding_Unit(
    input [4:0] idex_rs,      // Registro rs en etapa EX
    input [4:0] idex_rt,      // Registro rt en etapa EX
    input [4:0] exmem_rd,      // Registro destino en etapa MEM
    input [4:0] memwb_rd,      // Registro destino en etapa WB
    input exmem_RegWrite,      // Decide si MEM va a escribir en registros
    input memwb_RegWrite,      // Si WB va a escribir en registros o no

    output reg [1:0] ForwardA, // Control para el operando A de la ALU
    output reg [1:0] ForwardB  // Control para el operando B de la ALU
);

always @* begin
    // Valores por defecto: Usar los datos normales que vienen del Banco de Registros (00)
    ForwardA = 2'b00;
    ForwardB = 2'b00;

    // RIESGOS EN LA ETAPA EX
    // Si la instrucción en MEM va a escribir, el destino no es el registro 0,
    // y coincide con rs o rt de la instrucción actual en EX:
    if (exmem_RegWrite && (exmem_rd != 5'b0) && (exmem_rd == idex_rs)) begin
        ForwardA = 2'b10; // Adelantar desde la salida de la ALU (EX/MEM)
    end

    if (exmem_RegWrite && (exmem_rd != 5'b0) && (exmem_rd == idex_rt)) begin
        ForwardB = 2'b10; // Adelantar desde la salida de la ALU (EX/MEM)
    end

    // RIESGOS EN LA ETAPA MEM (Prioridad Baja)
    // Se evalúa como "else if" o con una condición extra para no pisar un riesgo EX activo
    if (memwb_RegWrite && (memwb_rd != 5'b0) &&
        !(exmem_RegWrite && (exmem_rd != 5'b0) && (exmem_rd == idex_rs)) &&
        (memwb_rd == idex_rs)) begin
        ForwardA = 2'b01; // Adelantar desde la etapa WB (salida del multiplexor MemToReg)
    end
end

```

```

end

if (memwb_RegWrite && (memwb_rd != 5'b0) &&
    !(exmem_RegWrite && (exmem_rd != 5'b0) && (exmem_rd == idex_rt)) &&
    (memwb_rd == idex_rt)) begin
    ForwardB = 2'b01; // Adelantar desde la etapa WB (salida del multiplexor MemToReg)
end
end

endmodule

```

```

1 //resolves data hazards (ej. Read-After-Write)
2 module Forwarding_Unit(
3     input [4:0] idex_rs,        // Registro rs en etapa EX
4     input [4:0] idex_rt,        // Registro rt en etapa EX
5     input [4:0] exmem_rd,        // Registro destino en etapa MEM
6     input [4:0] memwb_rd,        // Registro destino en etapa WB
7     input exmem_RegWrite,        // Decide s MEM va a escribir en registros
8     input memwb_RegWrite,        // Si WB va a escribir en registros o no
9
10    output reg [1:0] ForwardA,    // Control para el operando A de la ALU
11    output reg [1:0] ForwardB    // Control para el operando B de la ALU
12 );
13
14 always @* begin
15     // Valores por defecto: Usar los datos normales que vienen del Banco de Registros (00)
16     ForwardA = 2'b00;
17     ForwardB = 2'b00;
18
19     // RIESGOS EN LA ETAPA EX
20     // Si la instrucción en MEM va a escribir, el destino no es el registro 0,
21     // y coincide con rs o rt de la instrucción actual en EX:
22     if (exmem_RegWrite && (exmem_rd != 5'b0) && (exmem_rd == idex_rs)) begin
23         ForwardA = 2'b10; // Adelantar desde la salida de la ALU (EX/MEM)
24     end
25
26     if (exmem_RegWrite && (exmem_rd != 5'b0) && (exmem_rd == idex_rt)) begin
27         ForwardB = 2'b10; // Adelantar desde la salida de la ALU (EX/MEM)
28     end
29
30     // RIESGOS EN LA ETAPA MEM (Prioridad Baja)
31     // Se evalúa como "else if" o con una condición extra para no pisar un riesgo EX activo
32     if (memwb_RegWrite && (memwb_rd != 5'b0) &&
33         !(exmem_RegWrite && (exmem_rd != 5'b0) && (exmem_rd == idex_rs)) &&
34         (memwb_rd == idex_rs)) begin
35         ForwardA = 2'b01; // Adelantar desde la etapa WB (salida del multiplexor MemToReg)
36     end
37
38     if (memwb_RegWrite && (memwb_rd != 5'b0) &&
39         !(exmem_RegWrite && (exmem_rd != 5'b0) && (exmem_rd == idex_rt)) &&
40         (memwb_rd == idex_rt)) begin
41         ForwardB = 2'b01; // Adelantar desde la etapa WB (salida del multiplexor MemToReg)
42     end
43 end
44 end
45 endmodule
46
47

```

Módulo F_Unit en editor de texto.

BUFFERS

Buffer Instrucciones

```

module BufferInstr(

```

```

input clk,
input reset,
input flush, //PUERTO DE LIMPIEZA

input [31:0] instr_in,
input [31:0] pc4_in,

output reg [31:0] instr_out,
output reg [31:0] pc4_out
);

always @(posedge clk or posedge reset)
begin
    if(reset || flush) // SI HAY SALTO, METE UN NOP
    begin
        instr_out <= 32'b0;
        pc4_out <= 32'b0;
    end
    else
    begin
        instr_out <= instr_in;
        pc4_out <= pc4_in;
    end
end
endmodule

```

```

1 module BufferInstr(
2     input clk,
3     input reset,
4     input flush, //PUERTO DE LIMPIEZA
5
6     input [31:0] instr_in,
7     input [31:0] pc4_in,
8
9     output reg [31:0] instr_out,
10    output reg [31:0] pc4_out
11 );
12
13 always @(posedge clk or posedge reset)
14 begin
15     if(reset || flush) // SI HAY SALTO, METE UN NOP
16     begin
17         instr_out <= 32'b0;
18         pc4_out <= 32'b0;
19     end
20     else
21     begin
22         instr_out <= instr_in;
23         pc4_out <= pc4_in;
24     end
25 end
26 endmodule

```

Módulo BufferInstr en editor de texto.

Módulo buffer de datos

```

module BufferDatos(

```

```

input clk,
input reset,

input [31:0] pc4_in,
output reg [31:0] pc4_out,

input [31:0] rd1_in,
input [31:0] rd2_in,
input [31:0] imm_in,

input [4:0] rs_in,
input [4:0] rt_in,
input [4:0] rd_in,

input RegWrite_in,
input MemToReg_in,
input MemWrite_in,
input MemRead_in,
input ALUSrc_in,
input RegDst_in,
input Branch_in,

input [2:0] ALUOp_in,

input [5:0] funct_in,
input [4:0] shamt_in,
output reg [4:0] shamt_out,
output reg [5:0] funct_out,

output reg [31:0] rd1_out,
output reg [31:0] rd2_out,
output reg [31:0] imm_out,

output reg [4:0] rs_out,
output reg [4:0] rt_out,
output reg [4:0] rd_out,

output reg RegWrite_out,
output reg MemToReg_out,
output reg MemWrite_out,
output reg MemRead_out,
output reg ALUSrc_out,
output reg RegDst_out,
output reg Branch_out,

output reg [2:0] ALUOp_out
);

always @(posedge clk or posedge reset) begin
    if(reset) begin
        pc4_out    <= 32'b0;
        rd1_out    <= 32'b0;

```

```

    rd2_out    <= 32'b0;
    imm_out    <= 32'b0;
    rs_out     <= 5'b0;
    rt_out     <= 5'b0;
    rd_out     <= 5'b0;
    RegWrite_out <= 1'b0;
    MemWrite_out <= 1'b0;
    MemToReg_out <= 1'b0;
    MemRead_out <= 1'b0;
    ALUSrc_out <= 1'b0;
    RegDst_out <= 1'b0;
    Branch_out <= 1'b0;
    ALUOp_out  <= 3'b0;
    funct_out  <= 6'b0;
    shamt_out  <= 5'b0;
end
else begin
    pc4_out    <= pc4_in;
    rd1_out    <= rd1_in;
    rd2_out    <= rd2_in;
    imm_out    <= imm_in;
    rs_out     <= rs_in;
    rt_out     <= rt_in;
    rd_out     <= rd_in;
    RegWrite_out <= RegWrite_in;
    MemWrite_out <= MemWrite_in;
    MemToReg_out <= MemToReg_in;
    MemRead_out <= MemRead_in;
    ALUSrc_out <= ALUSrc_in;
    RegDst_out <= RegDst_in;
    Branch_out <= Branch_in;
    ALUOp_out  <= ALUOp_in;
    funct_out  <= funct_in;
    shamt_out  <= shamt_in;
end
end
endmodule

```

```

29 module BufferDatos(
30     input clk,
31     input reset,
32
33     input [31:0] pc4_in,
34     output reg [31:0] pc4_out,
35
36     input [31:0] rd1_in,
37     input [31:0] rd2_in,
38     input [31:0] imm_in,
39
40     input [4:0] rs_in,
41     input [4:0] rt_in,
42     input [4:0] rd_in,
43
44     input RegWrite_in,
45     input MemToReg_in,
46     input MemWrite_in,
47     input MemRead_in,
48     input ALUSrc_in,
49     input RegDst_in,
50     input Branch_in,
51
52     input [2:0] ALUOp_in,
53
54     input [5:0] funct_in,
55     input [4:0] shamt_in,
56     output reg [4:0] shamt_out,
57     output reg [5:0] funct_out,
58
59     output reg [31:0] rd1_out,
60     output reg [31:0] rd2_out,
61     output reg [31:0] imm_out,
62
63     output reg [4:0] rs_out,
64     output reg [4:0] rt_out,
65     output reg [4:0] rd_out,
66
67     output reg RegWrite_out,
68     output reg MemToReg_out,
69     output reg MemWrite_out,
70     output reg MemRead_out,
71     output reg ALUSrc_out,
72     output reg RegDst_out,
73     output reg Branch_out,
74
75     output reg [2:0] ALUOp_out
76 );
77
78 always @(posedge clk or posedge reset) begin
79     if(reset) begin
80         pc4_out    <= 32'b0;
81         rd1_out    <= 32'b0;
82         rd2_out    <= 32'b0;
83         imm_out    <= 32'b0;
84         rs_out     <= 5'b0;
85         rt_out     <= 5'b0;
86         rd_out     <= 5'b0;
87         RegWrite_out <= 1'b0;
88         MemWrite_out <= 1'b0;
89         MemToReg_out <= 1'b0;
90         MemRead_out <= 1'b0;
91         ALUSrc_out  <= 1'b0;
92         RegDst_out  <= 1'b0;
93         Branch_out  <= 1'b0;
94         ALUOp_out   <= 3'b0;
95         funct_out   <= 6'b0;

```

```

96         shamt_out    <= 5'b0;
97     end
98     else begin
99         pc4_out       <= pc4_in;
100        rd1_out        <= rd1_in;
101        rd2_out        <= rd2_in;
102        imm_out        <= imm_in;
103        rs_out         <= rs_in;
104        rt_out         <= rt_in;
105        rd_out         <= rd_in;
106        RegWrite_out   <= RegWrite_in;
107        MemWrite_out   <= MemWrite_in;
108        MemToReg_out   <= MemToReg_in;
109        MemRead_out    <= MemRead_in;
110        ALUSrc_out     <= ALUSrc_in;
111        RegDst_out     <= RegDst_in;
112        Branch_out     <= Branch_in;
113        ALUOp_out      <= ALUOp_in;
114        funct_out      <= funct_in;
115        shamt_out      <= shamt_in;
116    end
117 end
118 endmodule

```

Módulo BufferDatos en editor de texto.

Módulo Buffer Alu

```

module BufferALU(
    input clk,
    input reset,

    input [31:0] aluResult_in,
    input [31:0] writeData_in,

    input [4:0] writeReg_in,

    input Zero_in,
    input Branch_in,

    input RegWrite_in,
    input MemToReg_in,
    input MemWrite_in,
    input MemRead_in,

    output reg [31:0] aluResult_out,
    output reg [31:0] writeData_out,

    output reg [4:0] writeReg_out,

    output reg Zero_out,
    output reg Branch_out,

    output reg RegWrite_out,
    output reg MemToReg_out,
    output reg MemWrite_out,
    output reg MemRead_out

```

```

);

always @(posedge clk or posedge reset) begin
    if(reset) begin
        aluResult_out <= 32'b0;
        writeData_out <= 32'b0;
        writeReg_out <= 5'b0;
        Zero_out <= 1'b0;
        Branch_out <= 1'b0;
        RegWrite_out <= 1'b0;
        MemToReg_out <= 1'b0;
        MemWrite_out <= 1'b0;
        MemRead_out <= 1'b0;
    end
    else begin
        aluResult_out <= aluResult_in;
        writeData_out <= writeData_in;
        writeReg_out <= writeReg_in;
        Zero_out <= Zero_in;
        Branch_out <= Branch_in;
        RegWrite_out <= RegWrite_in;
        MemToReg_out <= MemToReg_in;
        MemWrite_out <= MemWrite_in;
        MemRead_out <= MemRead_in;
    end
end

endmodule

```

```

122 module BufferALU(
123     input clk,
124     input reset,
125
126     input [31:0] aluResult_in,
127     input [31:0] writeData_in,
128
129     input [4:0] writeReg_in,
130
131     input Zero_in,
132     input Branch_in,
133
134     input RegWrite_in,
135     input MemToReg_in,
136     input MemWrite_in,
137     input MemRead_in,
138
139     output reg [31:0] aluResult_out,
140     output reg [31:0] writeData_out,
141
142     output reg [4:0] writeReg_out,
143
144     output reg Zero_out,
145     output reg Branch_out,
146
147     output reg RegWrite_out,
148     output reg MemToReg_out,
149     output reg MemWrite_out,
150     output reg MemRead_out
151 );

```

```

153 always @(posedge clk or posedge reset) begin
154     if(reset) begin
155         aluResult_out <= 32'b0;
156         writeData_out <= 32'b0;
157         writeReg_out <= 5'b0;
158         Zero_out <= 1'b0;
159         Branch_out <= 1'b0;
160         RegWrite_out <= 1'b0;
161         MemToReg_out <= 1'b0;
162         MemWrite_out <= 1'b0;
163         MemRead_out <= 1'b0;
164     end
165     else begin
166         aluResult_out <= aluResult_in;
167         writeData_out <= writeData_in;
168         writeReg_out <= writeReg_in;
169         Zero_out <= Zero_in;
170         Branch_out <= Branch_in;
171         RegWrite_out <= RegWrite_in;
172         MemToReg_out <= MemToReg_in;
173         MemWrite_out <= MemWrite_in;
174         MemRead_out <= MemRead_in;
175     end
176 end
177
178 endmodule

```

Módulo BufferALU en editor de texto.

Módulo Buffer Final

```

module BufferFinal(
    input clk,
    input reset,

    input [31:0] memData_in,
    input [31:0] aluResult_in,

    input [4:0] writeReg_in,

    input RegWrite_in,
    input MemToReg_in,

    output reg [31:0] memData_out,
    output reg [31:0] aluResult_out,

    output reg [4:0] writeReg_out,

    output reg RegWrite_out,
    output reg MemToReg_out
);

always @(posedge clk or posedge reset) begin
    if(reset) begin
        memData_out <= 32'b0;
        aluResult_out <= 32'b0;
        writeReg_out <= 5'b0;
        RegWrite_out <= 1'b0;

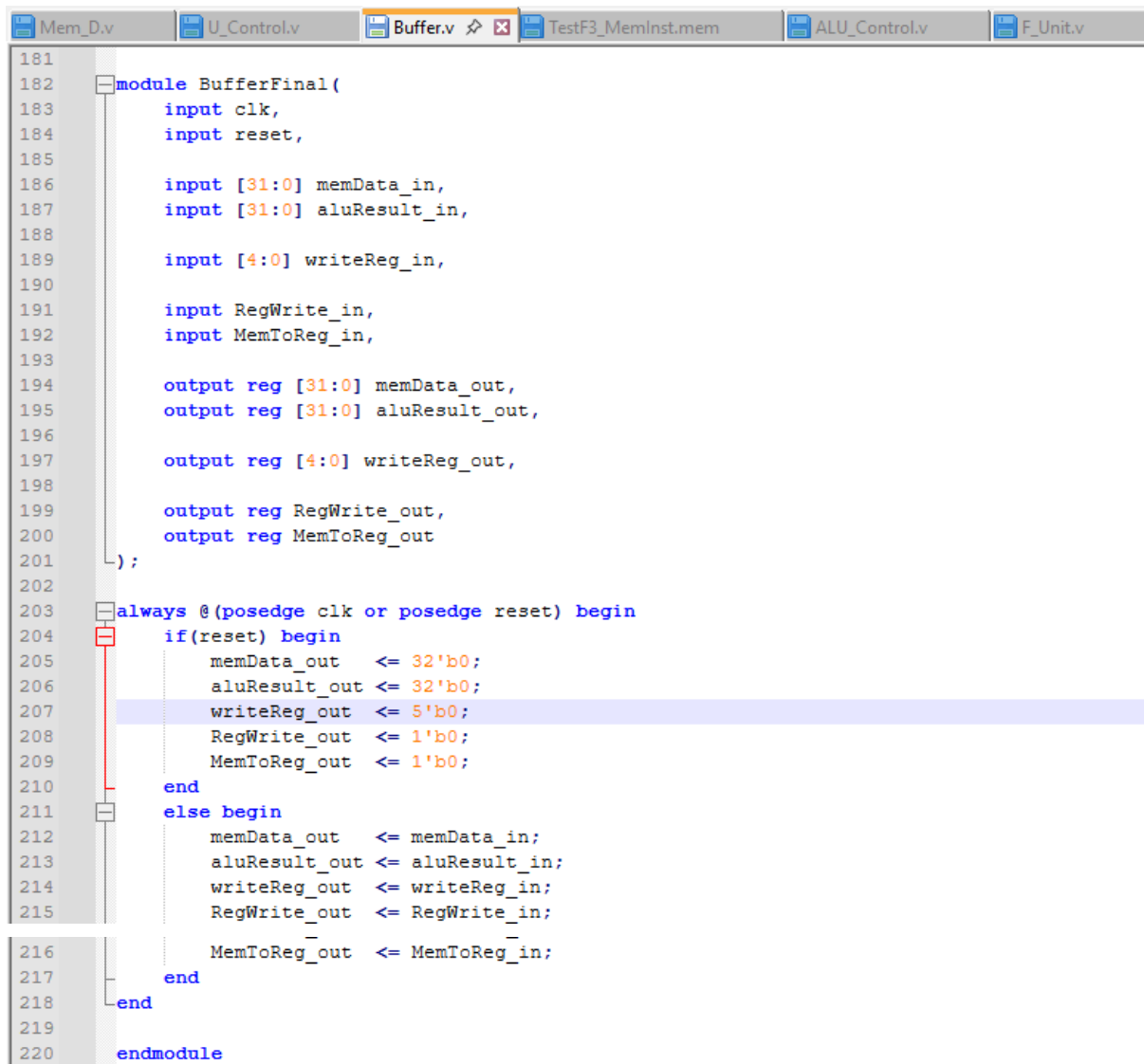
```

```

        MemToReg_out <= 1'b0;
    end
    else begin
        memData_out <= memData_in;
        aluResult_out <= aluResult_in;
        writeReg_out <= writeReg_in;
        RegWrite_out <= RegWrite_in;
        MemToReg_out <= MemToReg_in;
    end
end

endmodule

```



```

181
182 module BufferFinal(
183     input clk,
184     input reset,
185
186     input [31:0] memData_in,
187     input [31:0] aluResult_in,
188
189     input [4:0] writeReg_in,
190
191     input RegWrite_in,
192     input MemToReg_in,
193
194     output reg [31:0] memData_out,
195     output reg [31:0] aluResult_out,
196
197     output reg [4:0] writeReg_out,
198
199     output reg RegWrite_out,
200     output reg MemToReg_out
201 );
202
203 always @(posedge clk or posedge reset) begin
204     if(reset) begin
205         memData_out <= 32'b0;
206         aluResult_out <= 32'b0;
207         writeReg_out <= 5'b0;
208         RegWrite_out <= 1'b0;
209         MemToReg_out <= 1'b0;
210     end
211     else begin
212         memData_out <= memData_in;
213         aluResult_out <= aluResult_in;
214         writeReg_out <= writeReg_in;
215         RegWrite_out <= RegWrite_in;
216
217         MemToReg_out <= MemToReg_in;
218     end
219 end
220 endmodule

```

Módulo BufferFinal en editor de texto.

4.3 Ensamblador, Script Python y Código Máquina

Ensamblador (.asm)

```
# CONFIGURACION

addi $s0, $zero, 0    # base arreglo
addi $s1, $zero, 10    # n = 10
addi $s2, $zero, 45    # valor a buscar

# BUBBLE SORT

addi $t0, $zero, 0    # i = 0

CICLO_EXTERNO:

    addi $t8, $s1, -1
    slt $t9, $t0, $t8
    nop
    nop
    beq $t9, $zero, INICIO_BUSQUEDA
    nop
    nop

    addi $t1, $zero, 0 # j = 0

CICLO_INTERNO:

    sub $t7, $t8, $t0 # limite = n-1-i

    slt $t9, $t1, $t7
    nop
    nop
    beq $t9, $zero, FIN_INTERNO
    nop
    nop

    # offset = j * 4

    add $t4, $t1, $t1
    nop
    nop

    add $t4, $t4, $t4
    nop
    nop

    add $t5, $s0, $t4
    nop
    nop

    # A[j]
```

lw \$s3, 0(\$t5)

nop

nop

nop

nop

A[j+1]

lw \$s4, 4(\$t5)

nop

nop

nop

nop

if A[j+1] < A[j]

slt \$t6, \$s4, \$s3

nop

nop

beq \$t6, \$zero, NO_SWAP

nop

nop

swap

sw \$s4, 0(\$t5)

nop

nop

sw \$s3, 4(\$t5)

nop

nop

NO_SWAP:

addi \$t1, \$t1, 1

nop

nop

j CICLO_INTERNO

nop

nop

FIN_INTERNO:

addi \$t0, \$t0, 1

nop

nop

j CICLO_EXTERNO

nop
nop

BUSQUEDA BINARIA

INICIO_BUSQUEDA:

addi \$v0, \$zero, -1

addi \$t0, \$zero, 0 # izq
addi \$t1, \$s1, -1 # der

BUSQUEDA:

slt \$t2, \$t1, \$t0
nop
nop

bne \$t2, \$zero, FIN
nop
nop

medio = (izq + der)/2

add \$t3, \$t0, \$t1
nop
nop

srl \$t4, \$t3, 1
nop
nop

*# direccion = medio * 4*

add \$t5, \$t4, \$t4
nop
nop

add \$t5, \$t5, \$t5
nop
nop

add \$t6, \$s0, \$t5
nop
nop

leer A[medio]

lw \$s3, 0(\$t6)
nop
nop
nop

nop

encontrado

beq \$s3, \$s2, EXITO

nop

nop

A[medio] < X ?

slt \$t7, \$s3, \$s2

nop

nop

beq \$t7, \$zero, IZQUIERDA

nop

nop

derecha

addi \$t0, \$t4, 1

nop

nop

j BUSQUEDA

nop

nop

IZQUIERDA:

addi \$t1, \$t4, -1

nop

nop

j BUSQUEDA

nop

nop

EXITO:

add \$v0, \$zero, \$t4

nop

nop

FIN:

j FIN

nop

nop

Script de Python (Conversor)

```
import tkinter as tk
from tkinter import filedialog, messagebox
```

```
# — Tablas MIPS
```

```
REG = {
    "$zero": "00000", "$at": "00001", "$v0": "00010", "$v1": "00011",
    "$a0": "00100", "$a1": "00101", "$a2": "00110", "$a3": "00111",
    "$t0": "01000", "$t1": "01001", "$t2": "01010", "$t3": "01011",
    "$t4": "01100", "$t5": "01101", "$t6": "01110", "$t7": "01111",
    "$s0": "10000", "$s1": "10001", "$s2": "10010", "$s3": "10011",
    "$s4": "10100", "$s5": "10101", "$s6": "10110", "$s7": "10111",
    "$t8": "11000", "$t9": "11001",
}
```

```
# Tipo R – código de función (6 bits)
```

```
FUNCT = {
    "add": "100000", "sub": "100010", "and": "100100", "or": "100101",
    "xor": "100110", "slt": "101010", "srl": "000010", "sra": "000011",
    "tge": "110000", "teq": "110100",
}
```

```
# Tipo I – opcode (6 bits)
```

```
OPCODE_I = {
    "addi": "001000",
    "slti": "001010",
    "andi": "001100",
    "ori": "001101",
    "lw": "100011",
    "sw": "101011",
    "beq": "000100",
    "bne": "000101", # MODIFICACIÓN: Agregado bne
}
```

```
# Tipo J - opcode (6 bits)
```

```
OPCODE_J = {
    "j": "000010", # MODIFICACIÓN: Agregado opcode de saltos incondicionales
}
```

```
# — Helpers
```

```
def to_bin(val, bits):
    """Entero (con signo) a cadena binaria de `bits` dígitos."""
    v = int(val)
    if v < 0:
        # complemento a 2
        v = v & ((1 << bits) - 1)
    return format(v, f'0{bits}b')
```

```
def parse_mem(operand):
    """Descompone 'offset(base)' → (offset_str, base_reg_str)."""
    operand = operand.strip()
    if '(' in operand:
        offset_str, rest = operand.split('(', 1)
        base_reg = rest.rstrip(')')
        return offset_str.strip(), base_reg.strip()
    raise ValueError(f"Formato de memoria inválido: '{operand}'")
```

— Ensamblador de una línea

```
def ensamblar_linea(linea, num_linea=None, labels=None, pc_actual=None):
    """
```

Convierte una línea de ensamblador MIPS a una cadena de 32 bits.
Soporta resolución de etiquetas dinámicas en saltos de Tipo I y J.
"""

Limpieza básica

```
linea = linea.split('#')[0].replace(',', ' ').strip().lower()
```

```
if not linea:
```

```
    return None
```

```
partes = linea.split()
```

```
inst = partes[0]
```

```
if inst == "nop":
```

```
    return "0" * 32
```

```
opcode = "000000" # Tipo R por defecto
```

```
try:
```

```
    # — TIPO R: aritméticas/lógicas
```

```
    if inst in ("add", "sub", "and", "or", "xor", "slt"):
```

```
        rd, rs, rt = partes[1], partes[2], partes[3]
```

```
        return opcode + REG[rs] + REG[rt] + REG[rd] + "00000" + FUNCT[inst]
```

```
    # — TIPO R: desplazamientos
```

```
    elif inst in ("srl", "sra"):
```

```
        rd, rt, shamt = partes[1], partes[2], partes[3]
```

```
        return opcode + "00000" + REG[rt] + REG[rd] + to_bin(shamt, 5) + FUNCT[inst]
```

```
    # — TIPO R: trampas
```

```
    elif inst in ("tge", "teq"):
```

```
        rs, rt = partes[1], partes[2]
```

```
        return opcode + REG[rs] + REG[rt] + "00000" + "00000" + FUNCT[inst]
```

```
    # — TIPO I: aritméticas inmediatas
```

```
    elif inst in ("addi", "slti", "andi", "ori"):
```

```
rt, rs, imm = partes[1], partes[2], partes[3]
return OPCODE_I[inst] + REG[rs] + REG[rt] + to_bin(imm, 16)
```

— TIPO I: lw

```
elif inst == "lw":
    rt = partes[1]
    offset_str, rs = parse_mem(partes[2])
    return OPCODE_I["lw"] + REG[rs] + REG[rt] + to_bin(offset_str, 16)
```

— TIPO I: sw

```
elif inst == "sw":
    rt = partes[1]
    offset_str, rs = parse_mem(partes[2])
    return OPCODE_I["sw"] + REG[rs] + REG[rt] + to_bin(offset_str, 16)
```

— TIPO I: beq / bne (Saltos Condicionales)

MODIFICACIÓN: Cálculo de offset automático basado en etiquetas o números

```
elif inst in ("beq", "bne"):
    rs, rt, destino = partes[1], partes[2], partes[3]

    if labels and destino in labels:
        # El offset relativo en MIPS es: (PC_destino - (PC_actual + 1))
        offset_num = labels[destino] - (pc_actual + 1)
    else:
        offset_num = int(destino)

    return OPCODE_I[inst] + REG[rs] + REG[rt] + to_bin(offset_num, 16)
```

— TIPO J: j (Salto Incondicional)

MODIFICACIÓN: Agregado soporte completo para instrucciones Tipo J

```
elif inst == "j":
    destino = partes[1]
    if labels and destino in labels:
        target_address = labels[destino]
    else:
        target_address = int(destino)

    return OPCODE_J["j"] + to_bin(target_address, 26)
```

```
except KeyError as e:
    return f"ERROR línea {num_linea}: registro o campo desconocido {e} → '{linea}'"
except Exception as e:
    return f"ERROR línea {num_linea}: {e} → '{linea}'"

return f"ERROR línea {num_linea}: instrucción no reconocida → '{inst}'"
```

— GUI & PROCESAMIENTO

```

def seleccionar_archivo():
    ruta = filedialog.askopenfilename(
        filetypes=[("MIPS Assembly", "*.asm"), ("Text files", "*.txt")]
    )
    if ruta:
        entrada_ruta.delete(0, tk.END)
        entrada_ruta.insert(0, ruta)

def procesar_archivo():
    ruta_origen = entrada_ruta.get()
    if not ruta_origen:
        messagebox.showwarning("Atención", "Primero carga un archivo .asm")
        return
    try:
        with open(ruta_origen, 'r') as f:
            lineas_crudas = f.readlines()

        labels = {}
        lineas_instrucciones = []

        # — PRIMERA PASADA: Identificar y registrar posiciones de etiquetas —
        pc_temp = 0
        for idx, l in enumerate(lineas_crudas, start=1):
            linea_limpia = l.split('#')[0].replace(',', ' ').strip().lower()
            if not linea_limpia:
                continue

            # Procesar si la línea contiene una etiqueta (ej. "EXITO:")
            if ':' in linea_limpia:
                partes_etiqueta = linea_limpia.split(':', 1)
                nombre_label = partes_etiqueta[0].strip()
                labels[nombre_label] = pc_temp # Guarda la posición actual de memoria
                linea_restante = partes_etiqueta[1].strip()
                if not linea_restante:
                    continue # La línea solo tenía la etiqueta, pasamos a la siguiente
                linea_limpia = linea_restante

            lineas_instrucciones.append((idx, linea_limpia, pc_temp))
            pc_temp += 1

        # — SEGUNDA PASADA: Traducción real a binario de 32 bits —————
        resultado = []
        errores = []

        for num_linea, contenido, pc_actual in lineas_instrucciones:
            binario = ensamblar_linea(contenido, num_linea=num_linea, labels=labels,
pc_actual=pc_actual)
            if binario is None:
                continue
            if binario.startswith("ERROR"):
                errores.append(binario)
            else:

```

```

        resultado.append(binario)

    if errores:
        detalle = "\n".join(errores)
        if not messagebox.askyesno(
            "Advertencia",
            f"Se encontraron {len(errores)} error(es):\n\n{detalle}\n\n"
            "¿Deseas guardar de todas formas las instrucciones válidas?"
        ):
            return

    ruta_destino = filedialog.asksaveasfilename(
        defaultextension=".txt",
        filetypes=[("Text file", "*.txt"), ("Binary file", "*.bin")]
    )
    if ruta_destino:
        with open(ruta_destino, 'w') as f:
            # Tu código original separa la instrucción de 32 bits en 4 bloques de 8 bits
            lineas_8 = []
            for instruccion in resultado:
                for i in range(0, 32, 8):
                    lineas_8.append(instruccion[i:i+8])
                f.write("\n".join(lineas_8))

        messagebox.showinfo(
            "Éxito",
            f"Archivo generado con {len(resultado)} instrucción(es).\n{ruta_destino}"
        )

    except Exception as e:
        messagebox.showerror("Error", f"No se pudo procesar: {e}")

# — Ventana principal


---


root = tk.Tk()
root.title("Conversor – Algoritmos R + I + J")
root.geometry("540x270")
root.resizable(False, False)

tk.Label(root, text="Conversor ASM → Binario (R + I + J)",
        font=("Arial", 14, "bold")).pack(pady=12)

frame_carga = tk.Frame(root)
frame_carga.pack(pady=8)

btn_cargar = tk.Button(frame_carga, text="Cargar .asm",
        command=seleccionar_archivo, width=12)
btn_cargar.pack(side=tk.LEFT, padx=6)

entrada_ruta = tk.Entry(frame_carga, width=42)
entrada_ruta.pack(side=tk.LEFT, padx=6)

```

```

btn_convertir = tk.Button(
    root, text="CONVERTIR A BINARIO",
    bg="#4CAF50", fg="white", font=("Arial", 10, "bold"),
    command=procesar_archivo
)
btn_convertir.pack(pady=18, ipadx=12, ipady=6)

tk.Label(root,
    text="Soporta etiquetas de texto dinámicas (ej: CICLO_EXTERNO:, EXITO:)\n"
        "Tipo R: add sub and or xor slt srl sra teq tge nop | Tipo I: addi slti andi ori lw sw beq
        bne | Tipo J: j",
    font=("Arial", 8), fg="gray").pack()

root.mainloop()

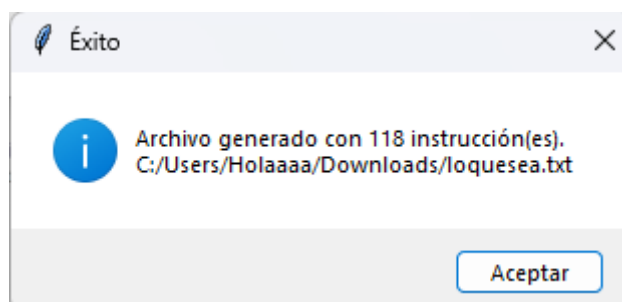
```



Interfaz del conversor en python.

Archivo en Binario (.txt)

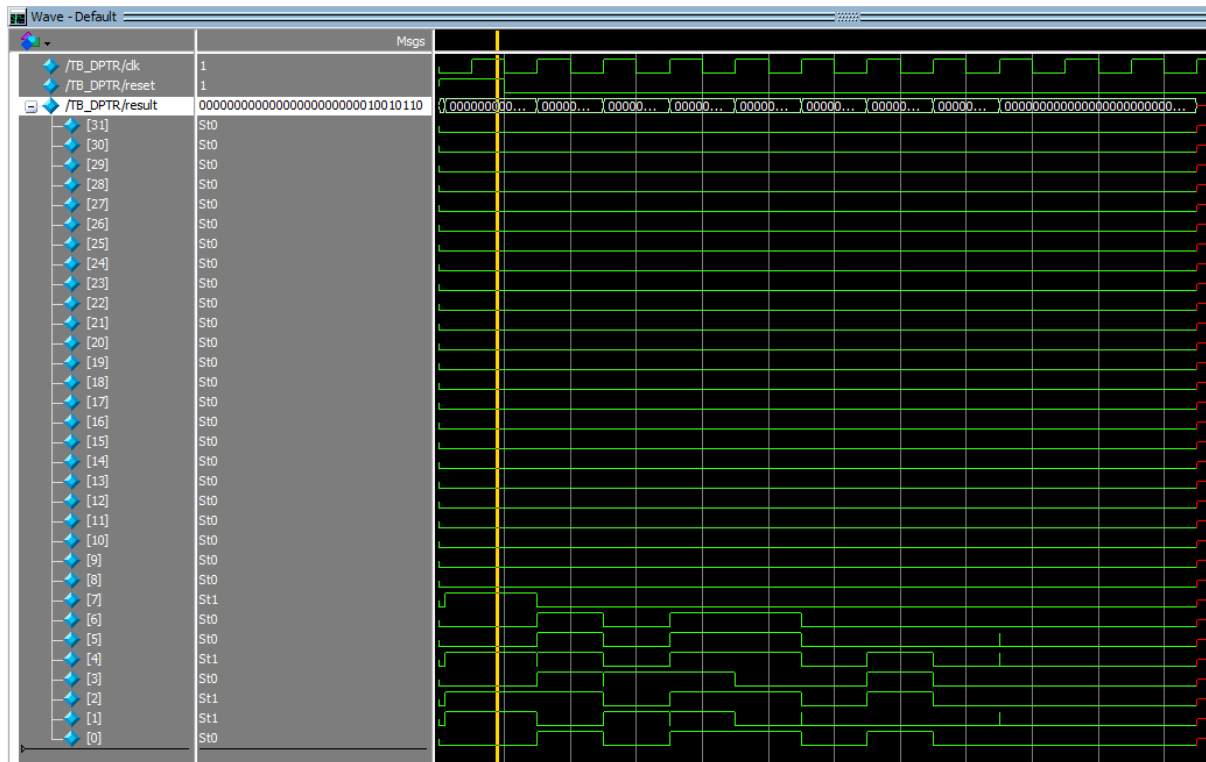
El código máquina es extenso para agregarlo aquí, pero al convertir el ASM a binario, nos da un total de 118 instrucciones, lo que equivale a las 472 líneas del .txt, es decir, 4 líneas de 8 bits cada una formando la instrucción de 32 bits. Es importante mencionar que la cantidad de instrucciones aumentó debido a los nop intercalando instrucciones, esto para el control de riesgos.



4.4 Resultados Simulación

Al correr la simulación, nuestro programa lee las instrucciones descritas en ensamblador (archivo .asm), las cuales fueron convertidas mediante el script de python, devolviéndonos por último el archivo .txt (se manda a llamar en el módulo Mem_I con readmemb) con las instrucciones en ceros y unos.

Luego con el módulo del test bench para el DPTR se le dio valor a los registros utilizados en las instrucciones tipo R del ensamblador para la fase 1.



Estas son las señales obtenidas en la ventana Wave para el TB.

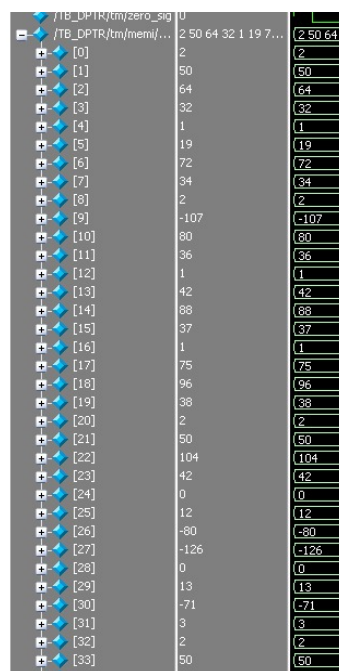
Memory Data - /TB_DPTR/tm/dp/BR/regs - Default		
0	0	0
2	0	0
4	0	0
6	0	0
8	150	125
10	10	127
12	117	0
14	0	0
16	0	100
18	50	25
20	30	10
22	29	0
24	0	0
26	0	0
28	0	0
30	0	0

Aquí puede observarse el memory list con los valores de cada registro después de correr la simulación, los valores corresponden a los resultados de cada operación.

Para la fase 2, nuestro procesador fue capaz de entender las instrucciones tipo I, luego de hacer modificaciones en nuestro data path, en la simulación se muestra cómo se guardan en la memoria.



Simulación en ModelSim donde se observan datos correctos en binario y decimal.



El procesador funcionó correctamente para la fase 2.

En el siguiente apartado se muestra cómo los buffers agregados desde la fase 2 funcionan correctamente, copiando cada valor para el siguiente bloque en cada ciclo de reloj.

00000000	00000004	00000008	0000000c	00000010
00000004	00000008	0000000c	00000010	00000014
02324020	01134822	02955024	012a5825	014b6026
00000000	02324020	01134822	02955024	012a5825

4.4.1 Simulaciones Finales

En la última fase, se lograron los resultados esperados para los dos puntos a desarrollar, es decir:

- Algoritmo propuesto como proyecto final de la materia:
 - Algoritmo de ordenamiento (Bubble Sort).
 - Búsqueda binaria.
- Estandarización de nuestro procesador para no solo leer nuestro conjunto de instrucciones dentro del algoritmo en ensamblador, sino también cualquier archivo que le entregemos en binario (usando para esto el plan de pruebas proporcionado por el maestro *TestF3_MemInst.mem*).

Simulación con Archivo de Prueba

Revisando el archivo *TestF3_MemInst.mem*, las instrucciones en hexadecimal quedan así (tomando cada 4 bytes como una instrucción MIPS):

Dirección (word)	Instrucción HEX	Ensamblador
0	0x00000000	nop
1	0x20080539	addi \$t0, \$zero, 1337
2	0x00000000	nop
3	0x08000000	j 0
4	0x00000000	nop
5	0x00000000	nop
6	0x2108FD2C	addi \$t0, \$t0, -724
8	0x08000012	j 18

Memory Data - /TB_DPTR/tm/dp/BR/regs - Default	
0	0
1	0
2	0
3	0
4	0
5	0
6	0
7	0
8	1337
9	0
10	0
11	0
12	0
13	0
14	0

Y aquí podemos observar el memory list de nuestro Banco de Registros, con el resultado correcto para la instrucción addi (1337 en el registro 8).

Simulación Algoritmo (Búsqueda Binaria)

Se realiza el algoritmo de búsqueda binaria en ensamblador, siendo necesario para esto tener el de ordenamiento, Bubble Sort, los resultados esperados para nuestra simulación son:

- **Memoria de datos:** mostrar el arreglo de datos (desordenado) con el que se cargó la memoria desde el test bench ordenados de menor a mayor, con un tamaño de $n = 10$, los datos deberían estar situados en las direcciones de memoria 0 a la 9.
- **Banco de registros:** el registro número 2 (\$v0) debe arrojar el resultado de la búsqueda binaria, para el caso de éxito (hallar en el arreglo ordenado el número 45) un número 7 (la posición) y para el caso de fracaso (que el 45 no se encuentre dentro del arreglo) un -1.

Además de los resultados esperados, hay otros registros que también deberían mostrar valores, los cuales se describen a continuación:

- Variables de Configuración Inicial
 - **Registro 16** (\$s0) = 0: La dirección base del arreglo en la memoria de datos (0).
 - **Registro 17** (\$s1) = 10: El tamaño del arreglo ($n = 10$).
 - **Registro 18** (\$s2) = 45: El valor que mandamos a buscar (45).
- El Rastro de la Última Iteración de la Búsqueda Binaria
 - **Registro 8** (\$t0 - izq) = 6
 - **Registro 9** (\$t1 - der) = 5

- **Registro 10** (\$t2) = 1: Este registro guarda el resultado de `slt $t2, $t1, $t0` (der < izq). Como $5 < 6$, el resultado es 1. Esto activó el salto `bne $t2, $zero, FIN`, rompiendo el ciclo correctamente.
 - **Registro 11** (\$t3) = 12: Este fue el cálculo de `izq + der`. Para dar 12, significa que en esa iteración tanto `izq` como `der` valían 6 ($6 + 6 = 12$).
 - **Registro 12** (\$t4 - medio) = 6: El cálculo de $12 \gg 1$ (desplazamiento a la derecha para dividir entre 2). El punto medio evaluado fue el índice 6.
 - **Registros 13 y 14** (\$t5 y \$t6) = 24: Es el cálculo de la dirección en bytes para el índice 6 ($6 * 4 = 24$). El procesador leyó exitosamente la posición `MEM[6]`.
- Residuos del Bubble Sort
 - **Registro 24** (\$t8) = 9: El límite inicial del ciclo externo ($n - 1 = 9$).
 - **Registros 19** (\$s3) y **20** (\$s4) = 50 y 15: Son los últimos dos valores que el Bubble Sort cargó de la memoria para comparar e intercambiar antes de finalizar el ordenamiento por completo.

Resultados

Address	Value
0	5
1	15
2	20
3	25
4	30
5	40
6	50
7	60
8	80
9	100
10	0
11	0
12	0
13	0
14	0
15	0
16	0
17	0
18	0
19	0
20	0
21	0
22	0
23	0

Memory list de la memoria de datos, se realizó de manera correcta el Bubble Sort, recibiendo como resultado el arreglo de manera ordenada, listo para la búsqueda binaria.

Index	Value 1	Value 2
0	0	0
2	-1	0
4	0	0
6	0	0
8	6	5
10	1	12
12	6	24
14	24	0
16	0	10
18	45	50
20	15	0
22	0	0
24	9	0
26	0	0
28	0	0
30	0	0

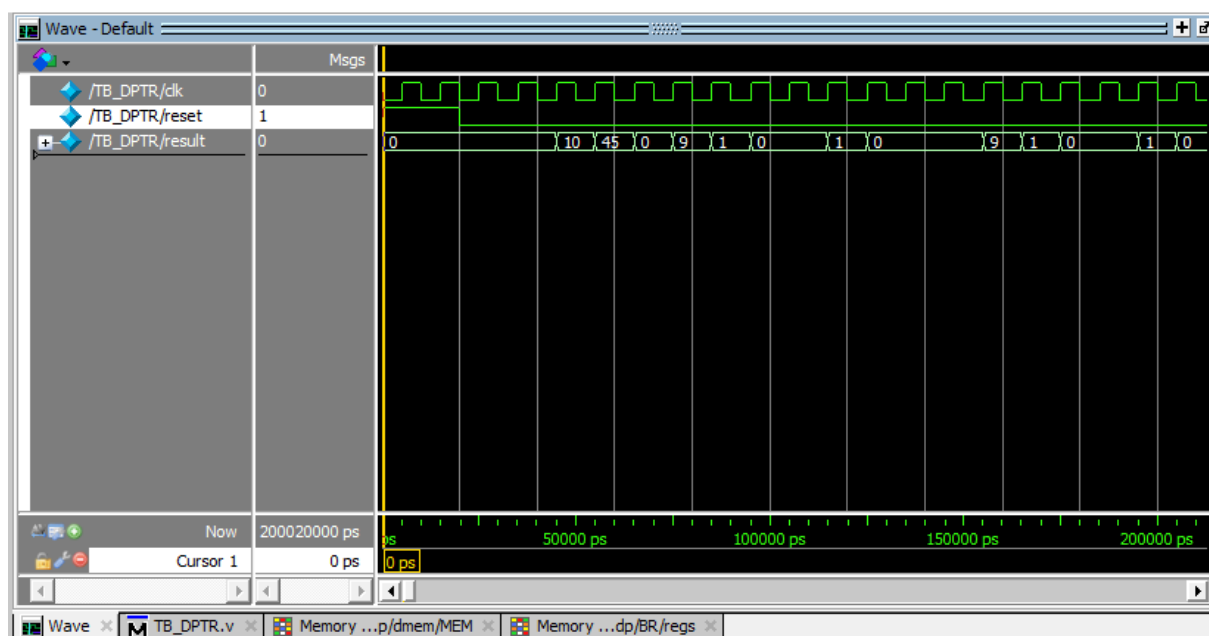
Memory list del Banco de Registros, lo más importante a notar es que en el registro 2 se encuentra el valor -1 (caso de fracaso).

Index	Value
0	5
1	15
2	20
3	25
4	30
5	40
6	45
7	60
8	80
9	100
10	0
11	0
12	0
13	0
14	0
15	0
16	0
17	0
18	0
19	0
20	0
21	0
22	0
23	0

Para el caso de éxito se inserta el valor 45 en el arreglo, y el Bubble Sort vuelve a funcionar.

Memory Data - /TB_DPTR/tm/dp/BR/regs		
0	0	0
2	6	0
4	0	0
6	0	0
8	6	6
10	0	12
12	6	24
14	24	1
16	0	10
18	45	45
20	15	0
22	0	0
24	9	0
26	0	0
28	0	0
30	0	0

Caso de éxito muestra el resultado 6 en el registro 2, es decir, la posición en que se encuentra el número que buscábamos, dirección número 6 de la memoria de datos.



Ventana Wave muestra las señales de cada instrucción que se va mandando, 0 (inicialización), 10 (tamaño del arreglo), 45 (número a buscar), etc.

5. Conclusión

Terminar esta tercera fase del proyecto y ver nuestro procesador MIPS ejecutando algoritmos como el Bubble Sort y la Búsqueda Binaria nos ayudó a entender, en la práctica, cómo funciona realmente la arquitectura de una computadora. Como equipo, logramos pasar de diseñar módulos sueltos a armar un procesador completo y funcional.

Hacer nuestro propio conversor en Python para pasar el código de ensamblador a binario fue un gran aprendizaje. Nos dimos cuenta de lo exactos que debemos ser para que la Unidad de Control entienda cada instrucción. Además, adaptar los algoritmos a las instrucciones de MIPS nos obligó a pensar como el hardware: aprendimos a cuidar el uso de los registros, a leer y guardar datos en la memoria correctamente, y a estructurar los saltos para que el programa no se perdiera.

Sin duda, la parte más difícil de todo el proyecto fue hacer funcionar el Pipeline de 5 etapas y solucionar los riesgos de datos (Hazards). Vimos directamente en las simulaciones cómo una instrucción que necesitaba un dato que todavía no estaba listo podía arruinar todo el resultado. Para arreglarlo, tuvimos que crear la Unidad de Adelantamiento (Forwarding Unit) y modificar el banco de registros para que pudiera leer y escribir en el mismo ciclo de reloj. Esto nos enseñó que, en el diseño de hardware, el tiempo y la sincronización lo son todo.

Finalmente, usar ModelSim para revisar las señales ciclo por ciclo fue clave para encontrar y corregir nuestros errores. Aprendimos a rastrear un problema desde la memoria hasta el cable exacto que estaba fallando. Al final, logramos hacer que todos los módulos trabajaran en equipo sin fallar. Cumplimos con los objetivos del proyecto y nos llevamos una experiencia muy valiosa sobre cómo se construye un procesador desde cero, lo cual nos deja una gran base para nuestra carrera.

6. Bibliografía

- Wikipedia. (2025). MIPS architecture. Wikipedia. Recuperado de https://en.wikipedia.org/wiki/MIPS_architecture
- GeeksforGeeks. (2024). Instruction format in MIPS. Recuperado de <https://www.geeksforgeeks.org/instruction-format-in-mips/>
- GeeksforGeeks. (2024). Conditional or ternary operator in programming. Recuperado de <https://www.geeksforgeeks.org/conditional-or-ternary-operator-in-programming/>
- Patterson, D. A., & Hennessy, J. L. (2017). Computer organization and design MIPS edition: The hardware/software interface (5th ed.). Morgan Kaufmann.
- Mano, M. M., & Ciletti, M. D. (2017). Digital design (6th ed.). Pearson.
- TutorialsPoint. (2024). Compiler design phases. Recuperado de https://www.tutorialspoint.com/compiler_design/compiler_design_phases.htm
- Nandland. (2023). Verilog conditional operator. Recuperado de <https://nandland.com/conditional-operator/>
- MIPS Technologies, Inc. (2016). MIPS32 Architecture For Programmers Volume I-A: Introduction to the MIPS32 Architecture. <https://www.mips.com>
- Lenovo. (s. f.). ¿Qué es MIPS?. Glosario de Lenovo. Recuperado el 1 de mayo de 2026, de <https://www.lenovo.com/es/es/glossary/mips/>
- MIPS Assembly/Instruction Formats - Wikibooks, open books for an open world. (n.d.). https://en.wikibooks.org/wiki/MIPS_Assembly/Instruction_Formats
- De Nihil, V. T. L. E. (2019, September 9). Introducción a la Arquitectura del MIPS. La Cripta Del Hacker. <https://lacriptadelhacker.wordpress.com/2019/09/09/introduccion-a-la-arquitectura-de-l-mips/>
- colaboradores de Wikipedia. (2024, December 6). Arquitectura en pipeline (informática). Wikipedia, La Enciclopedia Libre. [https://es.wikipedia.org/wiki/Arquitectura_en_pipeline_\(inform%C3%A1tica\)](https://es.wikipedia.org/wiki/Arquitectura_en_pipeline_(inform%C3%A1tica))
- CICS Transaction Server for z/OS. (2026, January 16). <https://www.ibm.com/docs/es/cics-ts/6.x?topic=concepts-pipelining>
- Startup, E. E. (2026, April 13). CPU Pipelining: lo que todo CTO debe saber – El Ecosistema Startup. El Ecosistema Startup. <https://ecosistemastartup.com/cpu-pipelining-lo-que-todo-cto-debe-saber/>